A Mechanized Library of Finite Game Theory: Kernel Games, Languages, and Bridges

Elazar Gershuni
elazarg@gmail.com

May 2026

Abstract

This chapter describes a Lean 4 library, built on top of mathlib [52], that formalizes finite game theory in a form suited to use as a downstream dependency. The library takes a deliberately uniform stance on what a game *is*: a *kernel game* consisting of per-player strategy types, an outcome type, a stochastic transition $K : \prod_i \mathsf{S}_i \to \mathrm{PMF}(\mathcal{O})$, and a utility $u : \mathcal{O} \to (\mathcal{I} \to \mathbb{R})$. Solution concepts are defined once on this carrier and on its utility-free shadow, the *game form*; concrete representations — normal-form games, extensive-form games, multi-agent influence diagrams, multi-round games, and factored-observation stochastic games — are presented as syntax-and-semantics packages that compile to a kernel game, and relationships between representations are stated as bridge theorems about the resulting kernel games. The classical theorems of finite non-cooperative game theory live in this layer: existence of mixed Nash equilibria via Brouwer on the product simplex, existence of correlated and coarse correlated equilibria, the minimax theorem, backward induction (Zermelo), the one-shot deviation principle, and Kuhn's behavioral/mixed equivalence. We discuss the design choices that make the library reusable — finiteness, discrete probability, the protocol/preference split, and the deviation-family schema for equilibrium notions — and we are explicit about what each choice gains and what it costs. The chapter is intended as the self-contained foundation on which more specialized theories (visibility-aware protocol semantics, mechanism design pipelines, multi-agent learning models) can be built.

Contents

Notation and glossary

We summarize the notation used throughout the chapter. Names match the Lean library's identifiers wherever the LaTeX rendering does not get in the way; a separate definitional supplement lists the exact Lean signatures.

Symbol	Meaning
$\mathcal{I}$	player index type, almost always a finite type
Payoff($\mathcal{I}$)	the type $\mathcal{I} \to \mathbb{R}$ of payoff vectors
PMF(X)	probability mass functions on X (discrete distributions)
S_i	strategy type of player i
σ	strategy profile: an element of $\prod_i \mathsf{S}_i$
$\sigma[i \mapsto s']$	unilateral deviation: replace σ_i by s'
K	outcome kernel $\prod_i \mathsf{S}_i \to \mathrm{PMF}(\mathcal{O})$
$\mathcal{F} = (\mathsf{S}, \mathcal{O}, K)$	game form: protocol layer
$\mathcal{G} = (\mathsf{S}, \mathcal{O}, u, K)$	kernel game: $\mathcal{F}$ with utility
toGameForm, toKernelGame	forgetful and adjoinment maps between $\mathcal{F}$ and $\mathcal{G}$
$\mathrm{EU}(\sigma, i)$	expected utility of player i under profile σ
$\succeq_i, \succ_i$	weak / strict preference of player i on PMF($\mathcal{O}$)
IsNash, IsBestResponse, etc.	solution-concept predicates
NFG, EFG, MAID, MR, FOSG	language-layer presentations
$\rightsquigarrow$	compilation arrow from a language to $\mathcal{G}$
$\cong, \Rightarrow, \sqsubseteq$	isomorphism, simulation, refinement (game-form level)

We freely use $\mathbb{N}$, $\mathbb{R}$, $\mathbb{R}_{\geq 0}$ for naturals, reals, and non-negative reals, and write δ_x for the Dirac (point-mass) distribution at x. "Finite" always means *Lean-finite*, i.e. a type with a `Fintype` instance, equivalently a type in bijection with $\{0, \ldots, n-1\}$ for some $n \in \mathbb{N}$.

1 Introduction

Game theory has long been a stable consumer of formal-mathematical notation, but its primary objects — strategy spaces, equilibria, mechanisms — are presented in textbooks [51, 59, 64, 79] in a mixture of styles that resists direct mechanization. A mechanization must commit, line by line, to whether "a game" means a tree, a payoff matrix, an information-set diagram, a stochastic process, or a Bayesian belief structure; whether "a strategy" is a function on histories, on information sets, or on observation sequences; whether "probability" lives in the discrete or the measure-theoretic world; and whether "equilibrium" is a predicate on a single profile, a profile distribution, or a correlation device. The textbook style routinely picks the most convenient answer locally, then quietly switches when the next theorem benefits from a different one.

This chapter describes a Lean 4 library that picks one answer at the bottom and pays the cost of plumbing every other notion through it. At the bottom is a finite, discrete-probability, utility-aware object — the *kernel game*. Every other representation in the library — normal-form, extensive-form, multi-agent influence diagrams, multi-round games, factored-observation stochastic games — is a *language* whose semantics is a kernel game. Solution concepts are defined once on the kernel-game (and game-form) layer and inherited by every language. Theorems about specific representations are stated either as theorems about their kernel games or as theorems about commuting compilations between languages.

1.1 Contributions

1. *A semantic core that is small enough to share.* The `KernelGame` record (§3) and its utility-free shadow `GameForm` are tiny: a strategy family, an outcome type, a stochastic kernel, and (for kernel games) a utility. Solution concepts are defined as predicates on this carrier and on `GameForm`,

parameterized by an arbitrary preference relation on outcome distributions. Every representation downstream — and every external consumer of the library — works against the same predicate schema.

2. *A protocol/preference split that pays for itself.* Following classical mechanism-design practice [59], we separate the strategy-and-outcome structure (the *game form*) from the payoff function. Lean lets us make the split structural rather than rhetorical: `GameForm` is a record without `utility`, and `KernelGame` = `GameForm` + `utility` via a definitional adjunction. This lets us state Kuhn's theorem, information-set machinery, and unilateral deviation as game-form results — independent of preferences — and instantiate Nash, dominance, correlated equilibrium, and Pareto efficiency for any preference relation on outcome distributions. Expected-utility preferences come for free as a special case.

3. *Languages and bridges as commuting compilations.* Each language layer (NFG, EFG, MAID, MultiRound, FOSG) ships with syntax, an SOS-style operational semantics, and a denotational `toKernelGame` that is the official semantic anchor. Bridges between languages are not merely translations between syntactic objects but theorems that two compilation paths to the same kernel game agree. This is what lets, for example, an EFG strategy theorem be transported to a MAID by composing two bridges, without re-proving anything strategic.

4. *Classical theorems, mechanized.* Existence of mixed Nash equilibria (via Brouwer on the product simplex, lifted from mathlib's fixed-point theory), of correlated and coarse correlated equilibria, the minimax theorem, the one-shot deviation principle, Zermelo's backward induction, and Kuhn's behavioral-vs-mixed-strategy equivalence are all stated and proved on the `KernelGame` / `GameForm` layer. Their statements live once; their consequences for any specific language layer are corollaries.

5. *An expressiveness layer.* Beyond raw compilation, the library exposes a *relation-parameterized expressiveness framework*: a comparison between two languages is a translation on syntax paired with a soundness theorem stating that the compiled kernel games agree under a chosen relation R (EU-isomorphism, simulation, refinement, etc.; §5.8). This is what gives substantive content to claims like "every finite NFG is a one-shot FOSG" or "every bounded MAID is an EFG up to strategic-form refinement," both of which are mechanized as R-reductions in this framework; comparisons that are not in the library (e.g., MAID directly to multi-round) appear as targets in §11.

6. *Mechanism-design and auction worked examples.* Bayesian games, a finite revelation-principle formulation, social-choice machinery, and auction fragments (Vickrey, first-price, all-pay, VCG) are formalized as consumers of the library. The auction layer is deliberately uneven: Vickrey and VCG carry dominant-strategy truthfulness theorems, first-price records a no-dominant-strategy result, and all-pay currently consists of format-specific payoff lemmas.

1.2 What this library is not

It is not a measure-theoretic foundation. All probability is discrete, encoded by mathlib's `PMF` [52] (we explain in §2 what is gained and lost by this choice). It is not a computational engine: many definitions are `noncomputable`, depending on classical choice for irrelevant case splits, and we are explicit about where a constructive implementation would diverge. It is not a coverage of continuous games: the theorem layer works under finite carrier hypotheses, and the core expectation operator

reduces to finite sums in those cases. It is, finally, not a stand-alone proof assistant for game-theoretic claims: it is a mechanized library, designed to be imported, depended on, and extended.

A separate *Definitional Supplement* accompanies this chapter. The supplement documents the Lean signatures of every major object and theorem with explicit textbook correspondence, so a reader who wants to verify that "what the library calls Nash equilibrium is the textbook Nash equilibrium" can do so by inspection. The chapter is the narrative; the supplement is the faithfulness certificate. Cross-references throughout this chapter point to the relevant supplement section.

1.3 Reading guide

Section 2 fixes the probabilistic and finiteness discipline and explains why we made the design choices we made. Section 3 introduces `GameForm`, `KernelGame`, their morphisms, and the deviation-family schema that all equilibrium notions instantiate. Section 4 catalogues the solution concepts. Section 5 introduces the language layer and its bridges. Section 7 states the major theorems and sketches their structure. Section 8 covers mechanism design and auctions. Section 9 collects the design tradeoffs we made explicit. Section 10 situates the library against prior formalization efforts and against the textbook tradition. Section 11 lists the natural extensions.

2 Preliminaries

This section fixes the probabilistic, type-theoretic, and finiteness conventions used throughout the chapter, and explains the most consequential design choices. We are deliberately verbose: each choice is paired with the alternative we rejected and a candid statement of what we gained and what we lost.

2.1 Discrete probability and `PMF`

Every distribution in the library is a probability mass function in mathlib's sense: a function $\mathrm{PMF}(X)$ assigning to each $x \in X$ a non-negative extended real number whose total mass equals one. The relevant API is monadic — $\mathrm{PMF}(\cdot)$ is the discrete Giry monad — with `pure`, `bind`, `map`, and a Dirac embedding $\delta_x \in \mathrm{PMF}(X)$. Expected value of a function $f : X \to \mathbb{R}$ under $\mu \in \mathrm{PMF}(X)$ is the (countable) sum $\sum_x \mu(x)\, f(x)$, formalized as `Math.Probability.expect`. When X is finite, this is a finite sum; when X is countably infinite, it is a tsum that converges by the unit-mass condition together with non-negativity of μ.

A *kernel* (in the library's sense, `Math.Probability.Kernel`) is a function $K : A \to \mathrm{PMF}(B)$. This is not a kernel in the measure-theoretic sense: no σ-algebra is involved. It is the discrete analogue, and is the right object for finite games: a strategy profile maps deterministically to a distribution over outcomes, and the no-measurability-needed nature of the discrete case makes it tractable to reason about by equational rewriting.

Why discrete, and what is gained. Mechanizing measure theory in dependent type theory is feasible — mathlib contains a substantial measure-theoretic library — but the cost is non-trivial. A measure on a space X requires a σ-algebra on X (a designated collection of subsets closed under complements and countable unions); functions of interest must be proved measurable; integration is defined first for non-negative measurable functions via simple-function approximation, then extended to integrable real-valued functions; Fubini–Tonelli, change of variables, and pushforward each require their own measurability hypotheses. Even small-looking statements acquire side conditions — "measurable function", "integrable function", "μ-a.e. equal" — that propagate through

every definition that uses them. In a finite or countable setting, these hypotheses are vacuously satisfied (every function from a finite type to a measurable space is measurable; every function into $\mathbb{R}$ from a finite type is integrable), but the *infrastructure* that produces the side conditions does not disappear: the proof obligations remain to be discharged, and the notation for distributions is more cumbersome.

In the discrete case the same obligations are vacuous and the machinery is unnecessary: when the underlying type is discrete, *every* subset is automatically measurable, *every* function out of it is automatically measurable, and integrals reduce to ordinary (possibly infinite) sums. But this only matters if we adopt the discrete formalism throughout — if the library is built on the measure-theoretic infrastructure, even the *statement* of an integral $\int f \, d\mu$ comes with a well-formedness side condition that f be measurable, and proofs that should rewrite by ordinary algebra acquire that side condition at every step. This is the cost we avoid.

By taking PMF($\cdot$) as the only distribution type, the library trades expressivity for ergonomics:

- *Gained.* Every distribution-level equation reduces to a finite-sum identity. Expected-utility identities follow by `simp`-style rewriting under the monadic laws. No measurability proof obligations arise. The "profile $\to$ distribution-over-outcomes" presentation of strategy spaces is direct. Pushforwards are given by `PMF.bind` and `PMF.map`; products by an explicit `pmfPi`; conditioning by ordinary support-restricted sums.

- *Lost.* Continuous strategy spaces (a key fragment of auction theory beyond the canonical models we treat), continuous type spaces in Bayesian games, and continuous-time stochastic games are out of scope. Some classical existence theorems — notably Glicksberg's existence of a mixed equilibrium for continuous payoff functions on compact strategy spaces [24], and existence of equilibria in Bayesian games with uncountable type spaces [55] — are not even statable in this idiom. Information-economic results that depend on absolute continuity of one distribution with respect to another are also unavailable.

Tradeoff. The library is intentionally a finite-discrete formalism. The boundary it places at PMF($\cdot$) is not a limitation we hope to remove later; it is the design contract. Downstream users who need continuous payoffs or uncountable type spaces must take the library as inspiration, not as a base, and reformulate the relevant definitions over mathlib's measure theory.

2.2 Finiteness

The library uses Lean's `Fintype` typeclass, which carries a finite enumeration of a type's elements. "Finite" means `Fintype`, equivalently: in bijection with $\{0, \ldots, n-1\}$ for some n. Most theorems require `Fintype` on the player type $\mathcal{I}$ and on each strategy type S_i; some require `DecidableEq` as well, e.g. everything that uses `Function.update` for unilateral deviation.

Decidability assumptions enter through `open Classical in`, which turns on the law of excluded middle for the duration of a single declaration. This is a benign use of classical reasoning when it is invoked only to make `Function.update` compute on abstract index types; on concrete finite types like `Fin n` the decision is constructive and the classical instance is definitionally redundant. We collect these uses in §9.5 and explain why they do not preclude a future computational refinement.

2.3 Type-theoretic conventions

A reader without a dependent-type-theory background will find several constructs in the library that look like ordinary set builders but mean something more precise. We gloss the ones that matter:

- *Dependent function types.* $\prod_i \mathsf{S}_i$ in ordinary mathematics is a product of sets indexed by $\mathcal{I}$. In Lean it is the dependent function type `(i : ι) → Strategy i`, written below as $\forall i, \mathsf{S}_i$. An element is a function that, for each i, returns a value in the set S_i — exactly a strategy profile. The library uses this throughout as the "strategy-profile" carrier.
- *Records / structures.* A record is a labeled product; `KernelGame ι` bundles four fields into a single value. Definitional unfolding of structure projections (e.g. "the `utility` field of $\mathcal{G}$ is u") is propositional in Lean and is freely used in proofs by simp-rewriting.
- *`Prop` vs. `Type`.* Propositions live in the sort `Prop` and are proof-irrelevant; data types live in `Type` (or higher universes) and are proof-relevant. Solution concepts are `Prop`-valued; `KernelGame` is `Type`-valued. The chapter uses "predicate" for a `Prop`-valued definition and "structure" for a `Type`-valued one.
- *Typeclass inference.* When we write that a definition "requires `Fintype` on S_i", we mean Lean's typeclass inference will look up an instance `Fintype` S_i from context. This is bookkeeping for the reader; the resulting object is the same whether the instance was synthesized or supplied explicitly.
- *Universe levels.* The library is universe-polymorphic in $\mathcal{I}$ and `Outcome` where it has to be, but the chapter treats them notationally as ordinary sets. Universe-level acrobatics arise only at one place — the lifting of solution concepts across reindexing along an equivalence — and we flag them when they do.

2.4 The mathlib base

The library depends on mathlib [52] for finite-type infrastructure, the simplex (`stdSimplex` and convex-combination machinery), product topology, fixed-point theorems (Brouwer in particular), the discrete Giry monad `PMF`, and a substantial amount of $\sum$-and-tsum API. Two non-trivial dependencies deserve explicit mention:

- *Brouwer's fixed-point theorem.* Mixed Nash existence (§7.1) is reduced to Brouwer applied to the product of standard simplices. Mathlib supplies the continuous-map and topology infrastructure; the Brouwer theorem itself is imported from the standalone `fixed-point-theorems-lean4` package [28], a Lean 4 mechanization of Brouwer (and Kakutani) for nonempty compact convex subsets of finite-dimensional normed spaces. We use it as a black box.
- *Discrete Giry monad laws.* Most equational reasoning in the library is monadic in `PMF`: bind associativity, left/right unit, and pushforward laws. These are mathlib lemmas we use as black boxes.

A small companion file `Math.Probability` adds the few discrete-probability constructions mathlib does not already contain (notably `Math.PMFProduct.pmfPi`, the independent product of a finite family of PMF(·)'s, used in the mixed extension of a kernel game).

2.5 What we mean by "finite game"

We will treat a finite game as a kernel game $\mathcal{G}$ with $\mathcal{I}$ finite, each S_i finite, and $\mathcal{O}$ finite. The language layer adds further structural finiteness — a finite extensive-form tree, finitely many information sets, a bounded horizon. All theorems in the chapter are stated under (some subset of) these assumptions, made explicit in each statement.

3 The semantic core: game forms and kernel games

The library is organized around two records — `GameForm` and `KernelGame` — that together act as the universal target into which every other representation compiles, and the universal domain on which every solution concept is defined. This section introduces them, the morphisms between them, and the deviation-family schema that unifies the equilibrium notions of §4.

3.1 Game forms

A *game form* over a player type $\mathcal{I}$ is a triple

$$\mathcal{F} \;=\; \big(\mathsf{S} : \mathcal{I} \to \mathsf{Type},\ \mathcal{O} : \mathsf{Type},\ K : \textstyle\prod_i \mathsf{S}_i \to \mathrm{PMF}(\mathcal{O})\big). \tag{1}$$

A *strategy profile* is an element of $\prod_i \mathsf{S}_i$, written σ. The kernel K assigns to each profile a distribution over outcomes; outcome distributions and correlated outcome distributions are derived in the obvious way:

$$\mathrm{outcomeDist}(\mathcal{F}, \sigma) \;=\; K(\sigma), \qquad \mathrm{correlatedOutcome}(\mathcal{F}, \mu) \;=\; \mu \gg\!= K$$

where $\mu \in \mathrm{PMF}(\prod_i \mathsf{S}_i)$ is a correlated profile distribution and $\gg\!=$ is the `PMF` bind. For a finite $\mathcal{I}$ with `DecidableEq`, unilateral deviation is the ordinary update operation

$$\sigma[i \mapsto s'] \;:=\; \sigma \text{ with coordinate } i \text{ replaced by } s'.$$

Two derived families of distributions formalize the kinds of deviation that equilibrium notions quantify over. The *constant* deviation family

$$\mathrm{constDev}(\mathcal{F}, \mu, i, s') \;=\; \mu \gg\!= \big(\sigma \mapsto \delta_{\sigma[i \mapsto s']}\big)$$

replaces i's coordinate by the same s' regardless of what μ sampled. The *recommendation-dependent* family

$$\mathrm{depDev}(\mathcal{F}, \mu, i, \mathrm{dev}) \;=\; \mu \gg\!= \big(\sigma \mapsto \delta_{\sigma[i \mapsto \mathrm{dev}(\sigma_i)]}\big)$$

lets the deviation be a function of i's recommendation; this is the family used in correlated equilibrium [6].

Design note. The two deviation families are kept as named records rather than inlined into each equilibrium notion. This is the single point at which the difference between Nash, coarse correlated equilibrium, and correlated equilibrium becomes visible: Nash and CCE quantify over constDev, CE quantifies over depDev. We exploit this in §4.1 by stating monotonicity results once on the schema.

Lean correspondence. `Core/GameForm.lean` for the record and derived operations; supplement §2.1 has the full Lean signatures.

3.2 Kernel games

A *kernel game* adds a utility to a game form,

$$\mathcal{G} \;=\; \big(\mathsf{S},\ \mathcal{O},\ u : \mathcal{O} \to \mathrm{Payoff}(\mathcal{I}),\ K\big), \tag{2}$$

where $\mathrm{Payoff}(\mathcal{I}) := \mathcal{I} \to \mathbb{R}$. The *expected utility* of player i under profile σ is

$$\mathrm{EU}(\mathcal{G}, \sigma, i) \;=\; \sum_{\omega} K(\sigma)(\omega)\; u(\omega)(i), \tag{3}$$

a finite sum under the finite-outcome hypotheses used by the major theorems. At the core API level the implementation is mathlib's unordered-sum expectation for PMF(·); finite-support lemmas recover the displayed formula. Expected utility is the only place where utility enters a numerical inequality.

The conversion toGameForm : $\mathcal{G} \mapsto \mathcal{F}$ forgets utility; the inverse toKernelGame$_u$: $\mathcal{F} \mapsto \mathcal{G}$ adjoins a utility u. These are mutual inverses up to record equality.

A kernel game over a finite outcome and finite-strategy domain admits an explicit *mixed extension*: each player's strategy type is replaced by PMF(S_i), and the outcome kernel samples a pure profile from the independent product $\bigotimes_i \sigma_i \in \mathrm{PMF}(\prod_i \mathsf{S}_i)$ and applies the original kernel. The mixed extension is itself a kernel game, indexed by the same $\mathcal{I}$, and serves as the carrier of mixed strategies in Nash existence (§7.1).

Lean correspondence. `Core/KernelGame.lean` (`KernelGame`, `KernelGame.eu`, `KernelGame.mixedExtension`, `reindex`); `Core/GameForm.lean` (`toGameForm`, `withUtility` and the round-trip lemmas); `Math/PMFProduct.lean` (`pmfPi`); supplement §2.2–2.4.

3.3 Solution concepts as predicates: the `*For` style

The library follows a uniform style for solution concepts that rewards a brief explanation, since it is unusual. Each concept exists in two forms:

- *EU-specific.* IsNash, IsBestResponse, WeaklyDominates, StrictlyDominates, IsCorrelatedEq, IsCoarseCorrelatedEq: predicates on $\mathcal{G}$ phrased directly in terms of the expected utility (3).
- *Preference-parameterized.* IsNashFor, IsBestResponseFor, WeaklyDominatesFor, StrictlyDominatesFor, IsCorrelatedEq$_{For}$, ...: predicates on $\mathcal{F}$ taking an additional argument, a preference relation $\succeq$: $\mathcal{I} \to \mathrm{PMF}(\mathcal{O}) \to \mathrm{PMF}(\mathcal{O}) \to \mathsf{Prop}$ on outcome distributions. "$\succeq_i\ d_1\, d_2$" means "player i weakly prefers distribution d_1 over d_2."

The two are tied together by lemmas of the shape

$$\mathcal{G}.\mathrm{IsNash}(\sigma) \iff \mathcal{G}.\mathrm{toGameForm}.\mathrm{IsNashFor}\big(\succeq_{\mathcal{G}}^{\mathrm{EU}},\, \sigma\big),$$

where $\succeq_{\mathcal{G},i}^{\mathrm{EU}}\ d_1\, d_2 \coloneqq \mathbb{E}_{d_1}[u(\cdot)(i)] \geq \mathbb{E}_{d_2}[u(\cdot)(i)]$, the expected-utility weak preference induced by $\mathcal{G}$.

Why two layers? Two reasons. First, several classical results — Kuhn's theorem, the existence of correlated equilibria, the dominance ⇒ Nash chain — are *purely structural*: they hold whenever players have a preorder on outcome distributions, regardless of whether that preorder comes from expected utility, lexicographic preferences, or maximin attitudes. Stating them at the $\mathcal{F}$.IsNashFor layer keeps them generic and stops them from masquerading as EU-specific theorems. Second, the EU-specific layer keeps the common case ergonomic: users who care only about expected utility never have to mention preference relations.

Why explicit-argument preferences and not a typeclass? This is a question worth posing directly, since at first glance a typeclass `[Preference G]` carrying the preference relation — with a default `euPref` instance — would make the EU-only user's syntax uniform (`G.IsNash` σ in every case). We considered this and rejected it because a substantial fraction of the library's theorems quantify over *more than one* preference simultaneously, and typeclasses fit such theorems poorly:

- Monotonicity (`IsNashFor.mono`, `IsDominantFor.mono`) takes two preferences $\succeq^{(1)}, \succeq^{(2)}$ together with a pointwise-implication hypothesis between them, and concludes a predicate for $\succeq^{(1)}$ from a predicate for $\succeq^{(2)}$.
- Pareto dominance is parameterized by a *pair* $(\succeq, \succ)$ of weak-and-strict preferences satisfying compatibility conditions; `ParetoDominatesFor pref spref` σ τ takes both explicitly.
- The transport theorem of §3.6 relates a preference on a source game form to its pullback along an outcome bijection — two preferences in the same statement, one a function of the other.
- Strict-from-weak / weak-from-strict bridges of §4.3 relate a strict preference to a weak preference under a hypothesis $\succ \Rightarrow \succeq$.

Each of these binds two preferences at once. A typeclass-based design would either force users to construct ad-hoc local instances for every multi-preference argument and switch between them by `let` bindings (noisy and error-prone), or fall back to explicit arguments for these theorems anyway (defeating the uniformity the typeclass was supposed to deliver). Explicit arguments are the natural shape from the start, and we add the EU-specific predicates as a parallel surface so that the EU-only ergonomic is recovered without invoking typeclass inference at all.

There is one place where the library does use a typeclass on preferences: `PrefPreorder pref` asserts that a preference relation is a preorder (reflexive and transitive). This is the correct use of a typeclass — it expresses a *property* of a given relation, not a choice between relations, and Lean's inference resolves it without any of the diamond/multi-preference issues above.

Tradeoff. The cost of the two-layer presentation is one extra indirection for non-EU users (they must instantiate the preference explicitly) and a handful of `_iff_` bridges to keep the EU-specific definitions in step with their preference-parameterized counterparts.

3.4 Morphisms, simulations, and isomorphisms

Game forms and kernel games each support a notion of structured map closed under composition with identities, with the standard identity- and associativity laws proved in the library. We treat them as ordinary categories of presentation but do not exploit further categorical structure in the chapter. At the game-form level, a *protocol map* from $\mathcal{F} \to \mathcal{F}'$ is a triple

$$(\text{stratMap}, \text{outcomeMap}, \text{preserved}) \quad \text{with} \quad \text{stratMap} : \forall i, \mathsf{S}_i \to \mathsf{S}'_i,\ \text{outcomeMap} : \mathcal{O} \to \mathcal{O}',$$

satisfying the *outcome-kernel preservation* law

$$K'(\text{stratMap} \circ \sigma) \;=\; K(\sigma) \gg\!= \big(\omega \mapsto \delta_{\text{outcomeMap}(\omega)}\big). \tag{4}$$

This is the discrete-Giry analogue of a coalgebra homomorphism, and is what makes a bridge between two languages strategically faithful (i.e. Nash equilibria pull back, dominance pulls back, etc., under hypotheses on stratMap). At the `KernelGame` level we add the utility-preservation condition $u'(\text{outcomeMap}(\omega))(i) = u(\omega)(i)$, yielding a *kernel-game simulation*, and call the morphism an *EU-isomorphism* when both stratMap and outcomeMap are bijective and the inverse is also a simulation.

Lean correspondence. Morphism records in `Core/GameMorphism.lean` (`ProtocolMap`, `KernelGame.Morphism`, with the `id_comp`/`comp_id`/`comp_assoc` laws); `Core/GameSimulation.lean` for kernel-game simulations and bisimulations; `Core/GameIsomorphism.lean` for isomorphisms; `Core/UtilityInvariance.lean` packages the standard transport results; `Concepts/EUProperties.lean` and `Concepts/OfEUProperties.lean` record EU structural properties and the `ofEU` round-trip. Supplement §2.6.

3.5 Reindexing and the player type

The player type $\mathcal{I}$ is left abstract throughout, but bridges between languages typically pick a canonical finite player type — most often $\mathrm{Fin}\, n$ for the cardinality of the source's player set. The library provides reindex : $(\mathcal{I} \simeq \kappa) \to \mathcal{G}_\kappa \to \mathcal{G}_\mathcal{I}$, transporting strategy spaces, utilities, and the outcome kernel along an equivalence. This is presentation bookkeeping, but it is the kind of bookkeeping that, omitted, leaks $\mathrm{Fin}\, n$ assumptions into every downstream consumer. The chapter writes $\mathcal{G}[e]$ informally for the reindexed game.

3.6 Transport of solution concepts under bridges

The use we make of protocol maps and EU-isomorphisms in §5.7 relies on a small family of transport theorems we record here for reference. Each is a direct calculation against the preservation law (4) and the definition of the relevant solution concept.

Theorem 3.1 (Solution-concept transport)**.** *Let $\mathcal{F}, \mathcal{F}'$ be game forms over the same player type $\mathcal{I}$ and let $f : \mathcal{F} \to \mathcal{F}'$ be a protocol map with* $\mathrm{stratMap}_i$ *bijective for every i and* outcomeMap *bijective. Let $\succeq'$ be a preference on* $\mathrm{PMF}(\mathcal{O}')$ *and let* $\succeq_i \; d_1\, d_2 :=\succeq'_i \; (\mathrm{outcomeMap}_*\, d_1)\, (\mathrm{outcomeMap}_*\, d_2)$ *denote its pullback along the outcome bijection. Then for every profile σ,*

$$\mathcal{F}.\mathrm{IsNashFor}(\succeq, \sigma) \iff \mathcal{F}'.\mathrm{IsNashFor}\big(\succeq', \mathrm{stratMap} \circ \sigma\big).$$

The analogous statements hold for IsBestResponseFor*,* IsDominantFor*,* WeaklyDominatesFor*,* StrictlyDominatesFor*,* IsCorrelatedEqFor*, and* IsCoarseCorrelatedEqFor*. Specializing $\succeq$ to expected utility under a kernel-game simulation that preserves utility yields the EU-specific transport laws.*

This is the operational content of contributions 3 and 5 in §1: a bridge between two languages, viewed as a protocol-preserving (resp. EU-preserving) map between the compiled game forms (resp. kernel games), automatically transports solution concepts. Concrete bridges (§5.7) instantiate the theorem with specific maps and inherit a Nash/dominance/correlated-eq preservation law for free.

Lean correspondence. `Core/UtilityInvariance.lean` and `Core/GameSimulation.lean`.

3.7 Designated facts

Three theorems on $\mathcal{F}$ deserve early visibility:

- *Dominance implies Nash*: for any preference $\succeq$, if every player's strategy in σ is $\succeq$-dominant, σ is a $\succeq$-Nash equilibrium.
- *Best-response characterization*: σ is $\succeq$-Nash iff each σ_i is a $\succeq$-best response against the rest.
- *Correlated $\Rightarrow$ coarse correlated*: every correlated equilibrium is a coarse correlated equilibrium.

None of these depends on EU; they hold for any preference preorder. The EU-specific versions are corollaries of the generic statements via the EU/preference iff-bridges of §3.3.

Lean correspondence. All three in `Core/GameForm.lean` (`dominant_is_nash_for`, `isNashFor_iff_bestResponseFor`, `IsCorrelatedEqFor.toCoarseCorrelatedEqFor`).

4 Solution concepts

This section catalogues the solution concepts the library formalizes. All of them sit on top of `KernelGame` or `GameForm`; concrete language layers (§5) inherit them via toKernelGame. The presentation is grouped by the *deviation family* the concept quantifies over, which is the underlying organizing principle of the library and makes several otherwise-disparate equivalences (correlated ⇒ coarse correlated, dominance ⇒ Nash, etc.) visible as instances of one schema.

4.1 The deviation-family schema

Fix a game form $\mathcal{F}$, a player i, and a status-quo profile distribution $\mu \in \mathrm{PMF}(\prod_j \mathsf{S}_j)$. Let $\succeq$ be a per-player weak preference on $\mathrm{PMF}(\mathcal{O})$ and let $\mathcal{D}_i = (D_i, \mathrm{deviate}_i)$ be a *deviation family* for i: a type D_i of deviations and a function $\mathrm{deviate}_i : \mathrm{PMF}(\prod_j \mathsf{S}_j) \to D_i \to \mathrm{PMF}(\prod_j \mathsf{S}_j)$. A profile distribution μ is $\mathcal{D}$-*equilibrium for* $\succeq$ if no player has a profitable deviation in $\mathcal{D}$:

$$\forall i, \forall d \in D_i. \quad \succeq_i \ \big(\mathcal{F}.\mathrm{correlatedOutcome}(\mu),\ \mathcal{F}.\mathrm{correlatedOutcome}(\mathrm{deviate}_i(\mu, d))\big). \tag{5}$$

The library identifies exactly two deviation families that matter for the standard solution-concept inventory:

- $\mathcal{D}^{\mathrm{const}}$: $D_i = \mathsf{S}_i$, with $\mathrm{deviate}_i(\mu, s')$ replacing i's coordinate by s' on every sample of μ.
- $\mathcal{D}^{\mathrm{dep}}$: $D_i = \mathsf{S}_i \to \mathsf{S}_i$, with $\mathrm{deviate}_i(\mu, \mathrm{dev})$ replacing i's coordinate s by $\mathrm{dev}(s)$ on every sample.

The four "profile-distribution" equilibria of the library are then schema instances:

$$\begin{array}{lcl} \mathrm{IsNashFor}(\sigma) & \equiv & \mathcal{D}^{\mathrm{const}}\text{-eq for } \delta_\sigma, \\ \mathrm{IsCoarseCorrelatedEqFor}(\mu) & \equiv & \mathcal{D}^{\mathrm{const}}\text{-eq for } \mu, \\ \mathrm{IsCorrelatedEqFor}(\mu) & \equiv & \mathcal{D}^{\mathrm{dep}}\text{-eq for } \mu. \end{array}$$

Several library-wide consequences follow:

- *Hierarchy.* $\mathcal{D}^{\mathrm{const}}$ embeds into $\mathcal{D}^{\mathrm{dep}}$ via constant deviation functions, so every $\succeq$-CE is a $\succeq$-CCE. Setting $\mu = \delta_\sigma$ recovers $\succeq$-Nash from $\succeq$-CCE. Thus pure Nash point masses are CCEs, and CE distributions are CCE distributions; CE and CCE are both predicates on profile distributions, while Nash is a predicate on point profiles.
- *Monotonicity in* $\succeq$. If $\succeq^{(2)}$ implies $\succeq^{(1)}$ pointwise, then any $\mathcal{D}$-equilibrium for $\succeq^{(2)}$ is a $\mathcal{D}$-equilibrium for $\succeq^{(1)}$. The library exposes this as `IsNashFor.mono` and `IsDominantFor.mono`.

Lean correspondence. Schema in `Concepts/Deviation.lean`; full Lean text in supplement §3.1.

4.2 Nash equilibrium and best response

The EU-specific Nash predicate is

$$\mathrm{IsNash}_{\mathcal{G}}(\sigma) \iff \forall i, \forall s' \in \mathsf{S}_i,\ \mathrm{EU}(\mathcal{G}, \sigma, i) \ \geq\ \mathrm{EU}(\mathcal{G}, \sigma[i \mapsto s'], i), \tag{6}$$

and the preference-parameterized variant on $\mathcal{F}$ is the schema instance with $\mu = \delta_\sigma$ and $\mathcal{D} = \mathcal{D}^{\mathrm{const}}$. A *best response* is a strategy that maximizes the expected utility against fixed opponents:

$$\mathrm{IsBestResponse}_{\mathcal{G}}(i, \sigma, s) \iff \forall s' \in \mathsf{S}_i,\ \mathrm{EU}(\mathcal{G}, \sigma[i \mapsto s], i) \ \geq\ \mathrm{EU}(\mathcal{G}, \sigma[i \mapsto s'], i), \tag{7}$$

and the standard equivalence

$$\mathrm{IsNash}_{\mathcal{G}}(\sigma) \iff \forall i,\ \mathrm{IsBestResponse}_{\mathcal{G}}(i, \sigma, \sigma_i)$$

holds at the $\mathcal{F}$.IsNashFor/IsBestResponseFor level for any preference and is propagated to the EU level by the bridge of §3.3.

A *strict Nash equilibrium* requires the inequality to be strict for every $s' \neq \sigma_i$. The library's `StrictNashProperties.lean` records the implication strict $\Rightarrow$ Nash, asymmetry under a strict preference that is incompatible with reverse weak preference, and uniqueness in ordinal-potential games.

4.3 Dominance

Two notions of dominance are formalized. *Weak dominance*: s weakly dominates t for player i when, for every opponent profile, i's payoff under s is $\succeq$-preferred to that under t. *Strict dominance* replaces $\succeq$ by a strict preference $\succ$ (typically the strict EU comparison).

A strategy s is *(weakly) dominant* for i when it weakly dominates every alternative; it is *strictly dominant* when the analogous statement holds for $\succ$. The library exposes these as both EU-specific (`IsDominant`, `IsStrictDominant`) and preference-parameterized (`IsDominantFor`, `IsStrictDominantFor`). The chain

$$\text{strict dominant} \ \Rightarrow\ \text{weak dominant} \ \Rightarrow\ \text{best response} \ \Rightarrow\ \text{Nash}$$

is recorded modularly: each step is one `simp`-able proposition, with no hidden EU dependence.

4.3.1 Iterated elimination and rationalizability

Iterated elimination of strictly dominated strategies is encoded as a sequence $\mathrm{Survives}_n$ of decreasing predicates: round 0 is true of every strategy, and round $n{+}1$ requires that the strategy both survived round n and is not strictly dominated, restricted to round-n survivors of all opponents. A strategy is *rationalizable* when it survives every round. The library proves:

- Survival is monotone decreasing in rounds.
- Every Nash strategy survives every round, hence is rationalizable.
- Every dominant strategy is rationalizable.
- In a *dominance-solvable* game, the unique surviving profile is Nash.

Lean correspondence. Predicates and theorems in `Concepts/Rationalizability.lean` (`Survives`, `IsNash.survives`, `IsNash.isRationalizable`, `IsDominant.isRationalizable`); the dominance-solvable case in `Concepts/DominanceSolvable.lean`.

4.4 Correlated and coarse correlated equilibria

Aumann's correlated equilibrium [6] interprets $\mu \in \mathrm{PMF}(\prod_i \mathsf{S}_i)$ as a public correlation device that recommends a private action $s = \mu_i$ to each player. The device is incentive-compatible when no player gains in expectation by ignoring the recommendation in favor of any function of it. This is exactly $\mathcal{D}^{\mathrm{dep}}$-equilibrium under EU. Coarse correlated equilibrium [31] weakens the deviation menu from arbitrary functions to constant strategies, which yields $\mathcal{D}^{\mathrm{const}}$-equilibrium under EU.

The library proves the standard inclusions

$$\delta(\succeq\text{-Nash}) \subseteq \succeq\text{-CCE}, \qquad \succeq\text{-CE} \subseteq \succeq\text{-CCE}.$$

Here `IsCorrelatedEqFor.toCoarseCorrelatedEqFor` records the CE $\Rightarrow$ CCE inclusion; the Nash-to-CCE statement is the schema instance $\mu = \delta_\sigma$. In the EU-specific layer, existence of correlated and coarse correlated equilibria for any finite $\mathcal{G}$ is the content of §7.2.

4.5 Pareto efficiency, social welfare, and price of anarchy

A profile σ *Pareto-dominates* τ for a pair $(\succeq, \succ)$ when every player weakly prefers σ's outcome distribution and at least one player strictly does; σ is *Pareto-efficient* when no profile dominates it. The EU specialization uses $\succeq^{\mathrm{EU}}$ and $\succ^{\mathrm{EU}}$. The library tracks the basic structural results — irreflexivity, asymmetry — modulo the irreflexivity and incompatibility hypotheses on $(\succeq, \succ)$.

Social welfare is the EU sum $\mathrm{SW}(\mathcal{G}, \sigma) = \sum_i \mathrm{EU}(\mathcal{G}, \sigma, i)$ over a finite player set. The *price of anarchy* [42, 69] compares the worst-case Nash welfare to the optimum,

$$\mathrm{PoA}(\mathcal{G}) \;=\; \frac{\sup_\sigma \mathrm{SW}(\mathcal{G}, \sigma)}{\inf_{\sigma \in \mathrm{Nash}(\mathcal{G})} \mathrm{SW}(\mathcal{G}, \sigma)}, \tag{8}$$

with the dual *price of stability* $\sup_\sigma \mathrm{SW} / \sup_{\sigma \in \mathrm{Nash}} \mathrm{SW}$. The library formalizes the welfare quantities the ratios are built from — optimal welfare, worst Nash welfare, best Nash welfare — plus the comparison lemmas between them. It does not yet package the ratio-valued PoA/PoS definitions; those require explicit positivity and non-empty-Nash hypotheses.

Lean correspondence. Welfare quantities in `Concepts/PriceOfAnarchy.lean` (`optimalWelfare`, `worstNashWelfare`, `bestNashWelfare`).

4.6 Security strategies and minimax value

Player i's *security level* [86] is the guarantee value over a worst-case opponent profile,

$$\mathrm{sec}(\mathcal{G}, i) \;=\; \sup_{s \in \mathsf{S}_i} \inf_{\sigma_{-i}} \mathrm{EU}\big(\mathcal{G}, \sigma_{-i}[i \mapsto s], i\big), \tag{9}$$

and a *security strategy* for i achieves this supremum. The library formalizes the worst-case-EU and security-level quantities, proves that a security strategy exists in the finite case, and proves that Nash payoffs are at least the security level. The two-player zero-sum theorem in §7.3 is stated through a mixed Nash profile with saddle-style guarantee inequalities.

Lean correspondence. Definitions and the IR/Nash bound in `Concepts/SecurityStrategy.lean` (`worstCaseEU`, `securityLevel`); individual-rationality interface in `Concepts/IndividualRationality.lean`.

4.7 Regret

Internal and external regret are recorded as positive parts of EU differences. External regret of σ for player i is

$$\mathrm{Reg}^{\mathrm{ext}}(\mathcal{G}, \sigma, i) \;=\; \max_{s' \in \mathsf{S}_i} \big(\mathrm{EU}(\mathcal{G}, \sigma[i \mapsto s'], i) - \mathrm{EU}(\mathcal{G}, \sigma, i)\big)^+; \tag{10}$$

internal regret replaces the constant s' by a function of σ_i and is the dual of the $\mathcal{D}^{\mathrm{dep}}$ deviation family that defines correlated equilibrium. The library proves the structural fact that a profile distribution μ has zero average external regret iff it is a CCE, and zero average internal regret iff it is a CE — the standard regret-equilibrium duality [31].

4.8 Approximate equilibria

An ε-*Nash equilibrium* relaxes the Nash inequality by an additive ε slack: every unilateral deviation is at most ε-profitable. Closure under ε-relaxation is recorded for the deviation-family schema, so the same definition applies to ε-CCE and ε-CE. We do not prove approximation theorems (e.g., LP-duality bounds for ε-CE in succinct games) in this layer; we expose the definitions as the natural targets for downstream complexity-theoretic work.

Lean correspondence. `Concepts/ApproximateNash.lean`.

4.9 Potential games

A function $\Phi : \prod_i \mathsf{S}_i \to \mathbb{R}$ is an *exact potential* [57] when, for every player i and unilateral deviation $\sigma \to \sigma[i \mapsto s']$,

$$\mathrm{EU}(\mathcal{G}, \sigma[i \mapsto s'], i) - \mathrm{EU}(\mathcal{G}, \sigma, i) \;=\; \Phi(\sigma[i \mapsto s']) - \Phi(\sigma); \tag{11}$$

it is an *ordinal potential* when the equality is replaced by an iff between the two sides being positive. The library proves:

- Every exact potential is an ordinal potential.
- Every (local or global) maximizer of an ordinal potential is Nash; hence finite ordinal potential games admit pure Nash equilibria as a direct consequence (any maximizer of Φ on the finite product $\prod_i \mathsf{S}_i$ is Nash).
- Best-response dynamics terminates in finitely many steps in a finite ordinal potential game.

Team games (§4.10) are an instance: they are exact potential games with $\Phi = u_{\text{any}}$.

Lean correspondence. `Concepts/PotentialGame.lean` (`IsExactPotential`, `IsOrdinalPotential`, `IsExactPotential.toOrdinal`, `IsOrdinalPotential.nash_of_maximizer`); `Concepts/PotentialFIP.lean` and `Concepts/PotentialWellFounded.lean` for the dynamics; `Concepts/PotentialTeam.lean` for the team-game corollary (`IsTeamGame.isExactPotential`).

4.10 Zero-sum, constant-sum, team, and symmetric games

A kernel game is *zero-sum* when $\sum_i u(\omega)(i) = 0$ for every ω, and *constant-sum* for an $\mathcal{O}$-independent constant. The library proves that the Nash equilibria of a two-player constant-sum game give each player guarantee inequalities against opponent deviations.

A kernel game is a *team game* when all players have identical utilities. Team games are exact potential games with $\Phi = u_{\text{any}}$; consequently every welfare optimum is a Nash equilibrium, and the price of stability is 1.

A kernel game is *symmetric* when permutations of the player type act trivially on outcome distributions and utility (modulo the same permutation). The library proves the structural lemmas that permuting a Nash profile by a symmetry yields a Nash profile, and that all players obtain equal EU at a symmetric profile. Existence of a *symmetric* mixed Nash equilibrium (i.e. a fixed point of the gain map within the symmetric subspace) is a classical corollary of mixed Nash existence [62]; the corollary is not packaged in the library and is listed in §11 as a natural next step. *Evolutionary stability* [53] is formalized as the appropriate strict-dominance condition on a symmetric two-player game.

Lean correspondence. `Concepts/ZeroSum.lean`, `Concepts/ConstantSum.lean`, `Concepts/ZeroSumNash.lean`, `Concepts/ConstantSumNash.lean`, `Concepts/TeamGame.lean`,

`Concepts/SymmetricGame.lean` (`isSymmetricEU_nash_perm`, `isSymmetricEU_eu_eq`), `Concepts/EvolutionaryStability.lean`.

4.11 Common knowledge and individual rationality

Two further behavioral concepts are formalized for downstream use. An event is *common knowledge* among a group of agents when everyone knows it, everyone knows everyone knows it, and so on for every finite depth of nested knowledge. Aumann's operator [7] captures this fixed point on a finite partition structure (each agent has an information partition over the state space; common knowledge of an event E at state s holds iff E contains the smallest set in the meet of all partitions that contains s). *Individual rationality*, the second concept, is a participation constraint: a profile is individually rational for player i when i's payoff is at least their security level (§4.6). Both are formal scaffolding for the mechanism-design layer (§8).

Lean correspondence. `Concepts/CommonKnowledge.lean`; `Concepts/IndividualRationality.lean`.

5 Languages and bridges

A *language* in this library is a syntactic class of game descriptions paired with a compiler $\rightsquigarrow$: Syntax $\to \mathcal{G}$ that fixes its denotational semantics. Equipping the syntax with an SOS-style operational semantics is optional but ergonomic: several theorems about a language are most naturally stated by induction on the SOS, and SOS-equivalence between source programs is then a structural certificate that they compile to the same kernel game.

The library exposes five languages — normal-form (NFG), extensive-form (EFG), multi-agent influence diagrams (MAID), multi-round games (MR, with stochastic and repeated specializations), and factored-observation stochastic games (FOSG) — and the relation-parameterized expressiveness framework that relates them (§5.8).

5.1 Normal-form games

The normal-form layer is the simplest: a normal-form game over a finite player type $\mathcal{I}$ and a finite action family A_i is the data

$$\mathrm{NFG} = \big(\mathcal{I},\, \{\mathsf{A}_i\}_i,\, u : (\textstyle\prod_i \mathsf{A}_i) \to \mathrm{Payoff}(\mathcal{I})\big),$$

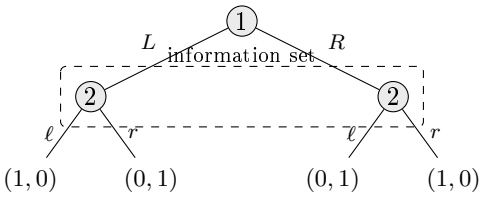
i.e., a function from joint pure actions to payoff vectors. The compilation NFG $\rightsquigarrow \mathcal{G}$ takes the strategy type to be A_i, the outcome type to be $\prod_i \mathsf{A}_i$, the kernel to be the Dirac evaluation $\sigma \mapsto \delta_\sigma$, and the utility to be the input function. The kernel-game image is deterministic; randomness in NFGs enters only through the mixed extension (§3.2).

The library's NFG layer also formalizes several worked subclasses as named structures: *congestion games* with Rosenthal's potential [68], *TU coalitional games* with the Shapley value [76], *Nash bargaining* [61], *stable matching* (predicates and structural lemmas) [21], *public goods*, and *Stackelberg leader-follower*. Each subclass states the content of its theorems at the level the library currently reaches: a congestion game is an exact potential game; the Shapley value comes with null-player, additivity, linearity, and symmetry lemmas; for stable matching, the predicates IsMatching, IsBlockingPair, IsStable and supporting structural lemmas — the Gale–Shapley algorithm and existence of a stable matching are listed in §11 as natural extensions. Each subclass is a direct consumer of the solution-concept catalogue (§4).

Lean correspondence. `Languages/NFG/` contains `Syntax`, `Compile`, plus the worked-subclass files `CongestionGame`, `CoalitionalGame`, `Bargaining`, `Matching`, `PublicGoods`, `Stackelberg`. Supplement §4.1.

5.2 Extensive-form games

An *extensive-form game* is a finite rooted tree whose non-terminal nodes are owned either by Nature (chance nodes with a probability distribution over children) or by a player (decision nodes from which the player chooses one child). Terminal nodes carry payoffs. Two pieces of textbook structure on top of this tree are crucial. First, the assignment of decision nodes to *information sets*: a partition of each player's decision nodes into equivalence classes the player cannot tell apart at play time. When a player moves at a node in an information set I, they know they are at some node of I, but not which one; their strategy must therefore commit to the same action across all nodes of I. This is what makes EFGs an imperfect-information formalism — perfect-information games are the special case where every information set is a singleton.



Player 1 moves first (L or R); player 2 moves second but without observing 1's choice — so 2's two decision nodes are in one information set (the dashed box), and 2's pure strategies are just $\{\ell, r\}$, not the four functions $\{L, R\} \to \{\ell, r\}$.

The library's EFG follows the textbook structure closely but factors it slightly differently for the type theory: rather than parameterize the tree by a player type directly, players are factored through an *information structure*:

$$\mathsf{InfoStruct} = \big(n,\, \mathsf{I} : \mathrm{Fin}\, n \to \mathsf{Type},\, \mathrm{arity} : \forall p\, I,\, \mathbb{N}_{>0}\big), \tag{12}$$

mapping each player $p \in \mathrm{Fin}\, n$ to a finite type of information sets and each information set to a positive action arity. A *game tree* over $\mathsf{InfoStruct}$ is the inductive type

$$\begin{aligned} \mathsf{Tree}(\mathsf{InfoStruct}, \mathcal{O}) \;:=\; & \mathsf{terminal}(z : \mathcal{O}) \\ & \mid \mathsf{chance}(k, \mu : \mathrm{PMF}(\mathrm{Fin}\, k), \mathrm{next} : \mathrm{Fin}\, k \to \mathsf{Tree}) \\ & \mid \mathsf{decision}(p, I : \mathsf{I}_p, \mathrm{next} : \mathrm{Fin}\,(\mathrm{arity}_p\, I) \to \mathsf{Tree}). \end{aligned} \tag{13}$$

A decision node stores only the information-set ID; the player identity is recovered from the structure. An EFG is a tree paired with a utility $\mathcal{O} \to \mathrm{Payoff}(\mathcal{I})$. Strategies come in three flavors:

- *Pure*: $\Sigma_p^{\mathrm{pure}} = \forall I : \mathsf{I}_p,\, \mathrm{Fin}\,(\mathrm{arity}_p\, I)$.
- *Behavioral*: $\Sigma_p^{\mathrm{beh}} = \forall I,\, \mathrm{PMF}(\mathrm{Fin}\,(\mathrm{arity}_p\, I))$.
- *Mixed*: $\Sigma_p^{\mathrm{mix}} = \mathrm{PMF}(\Sigma_p^{\mathrm{pure}})$.

The library's EFG semantics is *distributional*: given a behavioral profile $\beta : \forall p,\, \Sigma_p^{\mathrm{beh}}$, the function $\mathrm{evalDist}(\beta, t) \in \mathrm{PMF}(\mathcal{O})$ is defined by structural recursion on t, with chance nodes binding the chance distribution and decision nodes binding the player's behavioral distribution at the relevant

information set. The compilation EFG $\rightsquigarrow \mathcal{G}$ takes strategies to be behavioral profiles, outcomes to be the EFG's $\mathcal{O}$, the kernel to be evalDist, and the utility to be the EFG utility.

Design note. Distributional, not deterministic, semantics. An alternative would be to evaluate to a deterministic outcome under a pure profile and lift to mixed/behavioral profiles externally. We chose distributional semantics because it (a) fits chance nodes without ad-hoc lifting, (b) makes Kuhn's theorem (§7.5) a purely structural identity evalDist(β) = evalDist(realize(μ)) for the realization μ of β, and (c) fits the $K : \prod_i \mathsf{S}_i \to \mathrm{PMF}(\mathcal{O})$ shape of $\mathcal{G}$ without the strategies-to-outcomes adapter.

An *augmented* EFG in the FOSG-paper sense — a thin wrapper over an EFG that adds a public partition over all reachable histories and a per-player information partition over reachable histories, including those at which the player does not act — is also provided. Standard EFG information sets are only defined at decision nodes for the acting player; the augmentation lifts this to all of history space and is the carrier on which the FOSG/EFG bridge (§5.7) is most naturally stated.

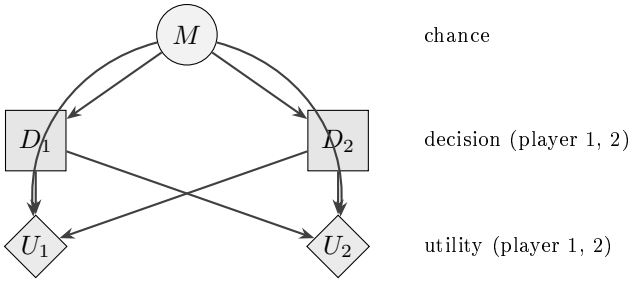
Lean correspondence. `Languages/EFG/` contains `Syntax`, `SOS`, `Compile`, `CompileObs` (compilation to ObsModelCore for Kuhn), `Augmented` (the augmented EFG), `Kuhn`, `Refinements` (subgame perfection, `EFGGame.IsSubgamePerfectEq`), `Examples`, `Tests`. Supplement §4.2.

5.3 Multi-agent influence diagrams

A *multi-agent influence diagram* (MAID, 40) represents a game as a directed acyclic graph whose nodes are typed as one of three kinds:

- *Chance nodes* X — random variables; each carries a conditional probability distribution (CPD) over its domain given the realizations of its parents. This is Nature's contribution.
- *Decision nodes* D — each owned by a specific player; the parents of a decision node represent the information the owning player observes when deciding, and the player chooses a value in the node's finite domain as a function of those parents.
- *Utility nodes* U — real-valued functions of their parents; each utility node is associated with a specific player, and that player's payoff is the sum of all their utility nodes.

The DAG structure makes causal and informational dependence *explicit* in the syntax: the parent set of a decision node is literally the information available to the deciding player, and the parent set of a utility node is literally the variables it depends on. Where an EFG has to flatten the same information into a tree with shared information sets, a MAID exposes the same content as a Bayesian network with decision and utility annotations.



A two-player MAID: a market state M is sampled by Nature; each player observes M and chooses an action D_i; each player's payoff U_i depends on the market state and both decisions. Decision nodes are squares, chance nodes are circles, utility nodes are diamonds.

The library formalizes a finite-domain fragment: each node has a finite domain, decision-node parents specify the player's information, and utility nodes are real-valued.

The compilation MAID $\rightsquigarrow \mathcal{G}$ proceeds via an intermediate *frontier evaluator*. At each step, the evaluator picks a node whose parents are all assigned and either (a) samples its chance distribution, (b) consults the player's strategy at the appropriate information set, or (c) accumulates its utility into the per-player running sum. The order of evaluation is non-trivial: when several decision nodes share a parent set, the evaluator's resolution must be order-independent to make the compilation well-defined. The diamond/commutation properties licensing the order-independent compiler are proved once; the same *frontier-stability* pattern reappears in the event-graph-based protocol layer that depends on this library [40].

Tradeoff. The order-of-evaluation issue is, in our view, the single largest design pressure in the MAID layer. An obvious alternative — fixing a topological order at compilation time — would simplify the proofs but build into the formalism a choice that no MAID-as-a-mathematical-object cares about. We accepted the proof obligations and got an order-independent compiler in exchange.

Lean correspondence. `Languages/MAID/` contains `Syntax`, `SOS`, `FrontierEval`, `FrontierLemmas` (the diamond properties), `FoldEval`, `Prefix`, `Compile`, `CompileObs`, `Kuhn`, `Theorems`. Supplement §4.3.

5.4 Multi-round games, repeated games, stochastic games

A multi-round game is a finite list of *rounds* [79], each round consisting of a state-dependent signal distribution (nature's joint signal), a per-player view function (state plus signal mapped to an observation), and a deterministic state transition keyed on the joint action profile. Strategies are pure and memoryless at the protocol level — a function from the player's view to an *optional* action — with mixed/behavioral and recall structure imposed externally as analysis-layer concerns.

The library's compilation MR $\rightsquigarrow \mathcal{G}$ generates the outcome kernel by folding the round list and threading the state through the chain of observations and transitions. A bounded horizon ensures termination. Two specializations are formalized on top of the multi-round skeleton:

- *Repeated games*: identical rounds, single payoff aggregation function (sum, discounted sum). This is the carrier for the standard formalizations of trigger strategies, tit-for-tat, and folk-theorem fragments. (A *folk theorem* says that any feasible, individually rational payoff vector can be sustained as an equilibrium average payoff of a sufficiently patient infinitely-repeated game; the classical Friedman / Fudenberg–Maskin versions [19, 20] are infinite-horizon and out of scope here.)
- *Stochastic games* [77]: rounds chosen by state, with a stochastic transition. Bounded horizon, by design.

The multi-round layer also compiles into the generic *InfoModel*/*ObsModel* substrate that the behavioral $\leftrightarrow$ mixed Kuhn theorem (§7.5) is stated against. This is the layer at which a per-round full-recall hypothesis becomes a per-round *ActionPosteriorLocal* hypothesis sufficient for the mixed-to-behavioral direction.

Lean correspondence. `Languages/MultiRound/` contains `Syntax`, `SOS`, `Compile`, `CompileObs` and its linearizations (`CompileObsLin`, `CompileObsLinAdequacy`, `CompileObsLinRecall`), `RepeatedGame`, `StochasticGame`, `Kuhn`, `Theorems`. Supplement §4.4.

5.5 Factored-observation stochastic games

To explain factored-observation stochastic games we have to distinguish two pre-existing notions. A *stochastic game* [77] is a Markov game on a finite state space: at each state, every active player simultaneously chooses an action; the joint action determines a stage payoff and a stochastic transition to a next state, and play continues to horizon. This formalism handles state and transition randomness cleanly but is opaque about *what each player observes* during a transition — the textbook treatment assumes either perfect monitoring of the state or a particular monitoring scheme named in prose. A *factored-observation stochastic game* (FOSG, 43) makes the monitoring explicit and *factored*: each transition produces a per-player *private* observation (independent component per player) and a single *public* observation visible to everyone. A player's strategy is then a function of their own observation *sequence* — the only data the player has access to.

Component	What it adds
state space + transition	"where am I, and where am I going?"
active-set per state	which players must move at each state
per-state available actions	legality of each player's action
reward vector	per-step payoff to each player
per-player private obs	what player i alone sees on a transition
public observation	what *every* player sees on a transition

The FOSG framework subsumes the standard formalisms used in the multi-agent reinforcement-learning literature (POMGs, Dec-POMDPs) by an appropriate choice of private and public observation channels, and it specializes to extensive-form games when the per-player observation sequence is rich enough to identify the current information set. This is the modern carrier for imperfect-information sequential games: a player's strategy can depend only on their own observation sequence, which is *exactly* the formal counterpart of an information set in the sequential setting.

In the library, an FOSG carries a state space, an active-set function, per-player available actions, a transition kernel, a reward vector, per-player private observations, and a public observation. Histories are recorded by an explicit history type. The finite-horizon FOSG kernel game is the canonical native carrier on which the bounded Kuhn theorem (§7.5) is proved.

Lean correspondence. `Languages/FOSG/` contains `Basic`, `History`, `Information`, `Strategy`, `Values`, `Execution`, `OutcomeClosure`, `Serial`, `Compile`, `Kuhn`, `Theorems`. Supplement §4.5.

5.6 Intrinsic-form games

A focused layer formalizes the *intrinsic* (a.k.a. Witsenhausen-style) representation of games following Heymann et al. [32]: rather than fixing an order in which agents act, the syntax exposes a set of agent decision *addresses* and information sets directly, and three strategy types — *mixed* (a distribution over a player's joint pure strategy space $\prod_{a \in A_p} \Lambda_a$), *product-mixed* (independent randomization over each address), and *behavioral* (independent randomization at each information set) — are defined and related propositionally. The unconditional product-mixed/behavioral equivalence is the intrinsic-form analogue of one direction of Kuhn's theorem (§7.5). The intrinsic layer is a separate syntactic root than EFG/MAID/MR/FOSG, used as a target by representations that prefer order-independent reasoning.

Lean correspondence. `Languages/Intrinsic/Strategies.lean` for the three strategy types and the `productMixedToBehavioral` / `behavioral_realizes_productMixed` equivalence; `PerfectRecall.lean` and `Theorems.lean` for the perfect-recall extension. Supplement §4.6.

5.7 Bridges as commuting compilations

A *bridge* between two languages L_1, L_2 is a translation $\tau : \text{Syntax}_{L_1} \to \text{Syntax}_{L_2}$ paired with a theorem stating that the two compilation paths agree under a specified semantic relation:

$$R\big(\, L_1.\rightsquigarrow(x),\ L_2.\rightsquigarrow(\tau(x))\,\big). \tag{14}$$

Different relations R correspond to different bridge strengths: EU-isomorphism (§3.4) is the strongest; protocol simulation (utility-preserving but not necessarily strategy-bijective) is intermediate; EU-equivalence-of-equilibria is the minimum useful target. The library exposes the following bridges:

Source	Target	Relation
NFG	EFG	EU-isomorphism (NFG as a depth-1 EFG)
NFG	FOSG	EU-isomorphism (NFG as a one-shot FOSG)
EFG	NFG	EU-equivalence on the realized strategic form
MAID	EFG	strategic-form refinement (information sets preserved)
EFG	FOSG	game-form simulation (FOSG view of an EFG with serial execution)

The bridge theorems are stated as identities on the compiled kernel games (or game forms), and are the glue that lets a theorem about, say, EFG behavioral strategies be transported to a MAID by composing the MAID→EFG bridge with the relevant strategic-form theorem on the EFG side.

Lean correspondence. `Languages/Bridges/` contains `NFG_EFG`, `NFG_FOSG`, `EFG_NFG`, `MAID_EFG`, and `FOSG/` (`ObsModelCore`, `AugmentedEFG`, `SerialExec`). Supplement §5.2.

5.8 Relation-parameterized expressiveness

The library packages the bridges into a generic expressiveness framework parameterized by a semantic relation R on $\mathcal{G}$.[1] A *language* in this layer is a pair (Syntax, $\rightsquigarrow$) with $\rightsquigarrow$ targeting $\mathcal{G}_{\mathcal{I}}$ for some fixed $\mathcal{I}$; a *reduction* $L \to M$ over a relation R is a translation τ and a proof that $R(L.\rightsquigarrow(x), M.\rightsquigarrow(\tau(x)))$ holds for every source x. Reductions compose under transitivity of R.

The relation taxonomy. The library catalogues the semantic relations this framework supports. They form a refinement order: a stronger relation determines a weaker one.

Relation	What it preserves
ProtocolEquivalent	Protocol isomorphism on game forms (strategies and outcomes relabeled bijectively, kernel preserved); ignores utility.
UtilityDistributionEquivalent	Bisimulation in $\mathcal{G}$: both protocol and joint utility distribution preserved at every profile.
UtilityDistributionSimulation	One-way version of the above; the directed reduction.
EUEquivalent	Equal expected utility for each player at corresponding profiles (the equilibrium-relevant relation).
EUSimulation	Directed EU-comparison; equilibria of the source pull back to the target.

[1] In the Lean code the relation, together with its `refl`/`symm`/`trans` proofs, is bundled in a record called `EquivalenceLens`; we mention the name only because the supplement uses it and so does any reader who reads the source. "Lens" here is the project's bundling vocabulary; it carries no connection to optics in functional programming or to lenses in category theory.

The bridges of §5.7 are reductions in this framework for the relation in their column. Context-aware extensions of the framework support languages whose compilation depends on a typing/visibility context — the natural target for downstream protocol-language formalizations.

Worked theorems. Two concrete worked expressiveness theorems are packaged in this form. An expressiveness claim like "every finite NFG is a one-shot FOSG up to EU-isomorphism" becomes the statement that NFG $\to$ FOSG is a reduction over the EU-isomorphism relation; "every bounded MAID is an EFG up to strategic-form refinement" becomes the analogous reduction over the refinement relation.

Design note. Expressiveness comparisons in the protocol-language literature are notoriously informal. By making the relation R explicit, the library forces every comparison statement to commit to which structural information is preserved and which is not. The resulting lattice of bridges (NFG, EFG, MAID, MR, FOSG ordered by which relation we can prove between them) is itself a contribution.

Lean correspondence. `Languages/Expressiveness/` contains `Basic` (`Language`, `Reduction`), `Relations` (the relation taxonomy), `CausalContext`, `MAID_EFG`, `EFG_FOSG`. Supplement §5.1.

6 A worked example end-to-end: Prisoner's Dilemma

To anchor the abstractions of §3–§5, we trace one small game through the full syntax-to-theorem pipeline. The example is deliberately the smallest non-trivial one — Prisoner's Dilemma in NFG form — and the corresponding Lean is in `Languages/NFG/Examples.lean`.

Source syntax. The player type is $\mathcal{I} = \text{Bool}$, the action type is

$$\mathsf{A} \;=\; \{\mathsf{cooperate}, \mathsf{defect}\},$$

shared between both players, and the payoff matrix is the classical one ($r_{CC} = 2$, $r_{DD} = 1$, $r_{DC} = 3$ for the defector and 0 for the cooperator). As an `NFGGame`, this is a record carrying the joint outcome type $\prod_{i:\text{Bool}} \mathsf{A}$, the identity outcome map, and the $\text{Bool} \times \mathsf{A} \times \mathsf{A} \to \mathbb{R}$ utility table.

Compilation to a kernel game. The NFG compiler (§5.1) sets the strategy type to A for each player, the outcome type to $\prod_i \mathsf{A}$, the kernel to the Dirac evaluation $\sigma \mapsto \delta_\sigma$, and the utility to the input table. The resulting `KernelGame` is deterministic; randomness enters only when we lift to its mixed extension (§3.2).

Solution-concept instantiation. The library's IsNash predicate (§4.2) on the compiled kernel game unfolds to

$$\forall i \in \{T, F\},\, \forall a' \in \mathsf{A}. \quad u_i(\sigma) \;\geq\; u_i(\sigma[i \mapsto a']),$$

which on Prisoner's Dilemma reduces to four inequality checks per profile. The Lean proof `pd_defect_is_nash`: `IsNashPure prisonersDilemma pd_defect_defect` is a single `intro/cases/cases/simp` that discharges the four inequalities by direct evaluation of the utility table. The corresponding "cooperate is not Nash" fact is the symmetric `linarith` after unfolding the deviation to defect.

Mixed-strategy extension and Nash existence. Lifting to the mixed extension produces a kernel game with strategy type PMF(A) for each player and outcome type $\prod_i \mathsf{A}$ as before. The library's mixed Nash existence theorem (Theorem 7.1) applies nonconstructively and returns some mixed equilibrium of the extension. Separately, the worked example proves that defect is dominant, so the pure (defect, defect) profile is Nash by the dominance⇒Nash chain (§4.3) without invoking Brouwer.

Bridging to FOSG. The NFG→FOSG bridge of §5.7 maps Prisoner's Dilemma to a one-shot FOSG with two world states ($\mathsf{init}, \mathsf{terminal}$), every player active at init, no observations, and a single deterministic transition delivering the NFG payoff as the FOSG reward. The bridge theorem `NFG_FOSG.NFGGame.toFOSG_terminal_iff` closes the expressiveness contract: the EU of any pure profile in the FOSG agrees with the EU in the NFG, so σ^* remains a Nash equilibrium under the EU-isomorphism transport (Theorem 3.1).

What this trace illustrates. *(i)* The NFG syntax is a ∼30-line Lean record; *(ii)* compilation to `KernelGame` is a definitional unfolding, not a proof; *(iii)* the strategic-form theorems (dominance⇒Nash; mixed existence; equilibrium preservation under bridges) apply as black boxes once the compilation has fixed the kernel game; *(iv)* the same game appears as a different syntactic object in two languages (NFG and FOSG) but with definitionally equal expected utility, so any equilibrium-level theorem proved on one of them transports to the other.

The same trace works for any NFG of finite cardinality, and a parallel trace for an EFG (e.g. a two-step centipede) is recorded in `Languages/EFG/Examples.lean`; we omit the latter here for space.

7 Major theorems

The library proves the central existence and structural theorems of finite non-cooperative game theory. The proofs are stated on the kernel-game/game-form layer (or on layered information-model abstractions; see §7.5), and inherited by every language via $\rightsquigarrow$. This section sketches their content; full proofs are in the cited Lean files.

7.1 Existence of mixed Nash equilibria

Theorem 7.1 (Mixed Nash existence; `KernelGame.mixed_nash_exists`). *Let $\mathcal{G}$ be a kernel game with $\mathcal{I}$ finite, every S_i finite and non-empty, and $\mathcal{O}$ finite. Then the mixed extension $\mathcal{G}^{\mathrm{mix}}$ admits a Nash equilibrium: a profile $\sigma^* \in \prod_i \mathrm{PMF}(\mathsf{S}_i)$ with*

$$\forall i,\ \forall \tau \in \mathrm{PMF}(\mathsf{S}_i).\quad \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma^*, i) \ \geq\ \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma^*[i \mapsto \tau], i).$$

Proof structure. Following Nash's original [60, 62], we define the gain map

$$\Phi_i(\sigma)(s) \ =\ \frac{\sigma_i(s) + (\mathrm{gain}_i(\sigma, s))^+}{1 + \sum_{s'} (\mathrm{gain}_i(\sigma, s'))^+}, \qquad \mathrm{gain}_i(\sigma, s) \ =\ \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma[i \mapsto \delta_s], i) - \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma, i),$$

which sends the product simplex $\prod_i \Delta(\mathsf{S}_i)$ continuously into itself. The Brouwer fixed-point theorem from the `fixed-point-theorems-lean4` package [28] is lifted to the product of standard simplices, and a fixed point of Φ is shown to be a Nash equilibrium by an algebraic argument bracketing the positive parts.

Why mathlib's Brouwer. The continuity content of the proof — that the Nash map is continuous on a compact convex polytope — is genuinely topological, and mechanizing the topology of the simplex from scratch would dwarf the rest of the proof. mathlib already provides a Brouwer-on-the-disk theorem and the infrastructure to lift it across homeomorphisms; we use the standard simplex's homeomorphism to a closed ball. The product of simplices step is handled by an explicit product-fixed-point lemma rather than by appealing to the product theorem in topology.

Tradeoff. Brouwer is non-constructive, so mixed Nash existence is `noncomputable`. A constructive existence proof would require either an explicit algorithmic construction (Lemke–Howson [48] for two-player games, or the support-enumeration class for finite games more generally) or a constructive fixed-point theorem on a finite simplicial structure (Sperner's lemma [81] suffices for two-player games but the multi-player extension is not as clean). We took the topological route because an existing Lean 4 mechanization of the Brouwer fixed-point theorem was available as the standalone `fixed-point-theorems-lean4` package [28], which proves Brouwer for nonempty compact convex subsets of finite-dimensional normed spaces via a cubical Sperner's-lemma route (Kuhn 1960); this gave us a ready dependency in the right shape for arbitrary n players and let us focus on the strategic-form content of the proof. Algorithm-level Nash existence is independent enough to live in a downstream module.

Lean correspondence. Statement `KernelGame.mixed_nash_exists` in `Theorems/NashExistenceMixed.lean`; the product-simplex Brouwer wrapper in `Concepts/ProductSimplexBrouwer.lean`. Supplement §6.1.

7.2 Existence of correlated and coarse correlated equilibria

Theorem 7.2 (CE / CCE existence; `KernelGame.correlatedEq_exists`, `coarseCorrelatedEq_exists`). *Under the same finiteness hypotheses as Theorem 7.1, $\mathcal{G}$ admits a correlated equilibrium and a coarse correlated equilibrium.*

Proof. The independent product distribution $\bigotimes_i \sigma_i^*$ over the mixed Nash profile σ^* of Theorem 7.1 is a correlated equilibrium of the original $\mathcal{G}$ (every recommendation-dependent deviation factors through a constant-strategy unilateral deviation in $\mathcal{G}^{\mathrm{mix}}$). CCE existence is the corollary $\succeq$-CE $\subseteq \succeq$-CCE (§4.4).

Tradeoff. This route gives CE existence as a corollary of mixed Nash existence and is the cleanest path through the finite-game machinery already in the library. Hart and Mas-Colell's regret-matching no-regret learning construction [31] produces CCE existence constructively without going through Brouwer. We have not formalized regret-matching itself, but the existence theorem is still available as a corollary of Nash existence; if a future constructive Nash-existence proof becomes available the dependency inverts trivially.

Lean correspondence. `KernelGame.correlatedEq_exists` and `coarseCorrelatedEq_exists` in `Theorems/CorrelatedEqExistence.lean`. Supplement §6.2.

7.3 The minimax theorem

Theorem 7.3 (Von Neumann's minimax; `KernelGame.von_neumann_minimax`). *Let $\mathcal{G}$ be a finite two-player zero-sum kernel game. There exist a real value v and a mixed Nash profile σ of $\mathcal{G}^{\mathrm{mix}}$ such that player 1 cannot push player 0's payoff below v by deviating from σ_1, and player 0 cannot push their payoff above v by deviating from σ_0:*

$$\mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma, 0) = v, \qquad \forall s_1,\ \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma[1 \mapsto s_1], 0) \geq v,$$

$$\forall s_0,\ \mathrm{EU}(\mathcal{G}^{\mathrm{mix}}, \sigma[0 \mapsto s_0], 0) \le v.$$

Moreover, all mixed Nash equilibria of a two-player zero-sum game give player 0 *the same expected utility.*

Proof. Theorem 7.1 gives existence of a mixed Nash equilibrium. In a two-player zero-sum game, mixed Nash profiles give saddle-style inequalities for $\mathrm{EU}(\cdot, 0)$, which combine with the zero-sum-Nash equality lemma and mixed Nash existence to deliver the displayed guarantees. The classical max-min/min-max equality is the standard mathematical reading of the finite saddle inequalities; the Lean theorem is stated in the guarantee form above.

Lean correspondence. Statement `KernelGame.von_neumann_minimax` in `Theorems/Minimax.lean`; supporting zero-sum-Nash equality in `Concepts/ZeroSumNash.lean` (`IsZeroSum.nash_eu_eq`). Supplement §6.3.

7.4 Backward induction (Zermelo) and the one-shot deviation principle

A *subgame-perfect equilibrium* [73] (SPE) of an extensive-form game is a strategy profile that is a Nash equilibrium not just of the whole game but of every subgame: the restriction of σ to any decision node and its descendants is itself Nash on the subtree rooted there. This rules out "empty threats" — Nash profiles that prescribe a player to do something off-path that they would not actually want to do if the off-path subgame were reached. In a finite perfect-information tree, SPE is the standard refinement of Nash equilibrium, and is the equilibrium concept the theorems below characterize.

Theorem 7.4 (One-shot deviation principle)**.** *Let G be a finite perfect-information extensive-form game. A pure profile σ is a subgame-perfect equilibrium of G iff it has no profitable one-shot deviation: at every reachable decision node, the action prescribed by σ yields at least as much expected utility (holding the rest of σ fixed) as any alternative.*

Theorem 7.5 (Zermelo's backward induction)**.** *Every finite perfect-information extensive-form game has a pure subgame-perfect equilibrium.*

The backward induction itself constructs, by recursion on the tree, a profile with no profitable one-shot deviation; the ODP closes the loop to subgame-perfection. Two non-trivial ingredients are worth flagging:

- *Disjointness of subtrees in perfect-info trees*: in a perfect-info tree, sibling subtrees have disjoint information-set supports. This is what allows backward induction to fix actions at a decision node without breaking optimality elsewhere.
- *Reachable-decision quantification*: the ODP's hypothesis quantifies over reachable decision nodes only. An explicit reach-by relation makes this precise and lets the proof ignore unreachable nodes whose action choice cannot affect EU.

Tradeoff. Backward induction is constructively well-defined on a finite tree, but the library's Zermelo is `noncomputable` because intermediate steps invoke classical reasoning to compare real-valued utilities. A computable variant restricting to rational utilities would compile but is not packaged.

Lean correspondence. Statements `EFG.oneShotDeviation_iff_spe` and `EFG.zermelo` in `Theorems/OneShotDeviation.lean` and `Theorems/Zermelo.lean`; the subtree disjointness lemma is `perfectInfo_disjoint_subtrees`; the reachability relation is `EFG.ReachBy`. Supplement §6.4.

7.5 Kuhn's theorem: behavioral ↔ mixed strategies

Kuhn's theorem [46] relates two carriers of randomness in extensive-form games: a *mixed strategy* is a distribution over pure strategies (a single global lottery), while a *behavioral strategy* chooses an independent local lottery at each information set. The two are payoff-equivalent under recall conditions; without recall, the asymmetry is genuine.

The library proves Kuhn's theorem on a representation-neutral substrate, the *observation model*: a tuple $(\sigma, \mathrm{Obs}, \mathsf{A})$ recording the world state, each player's observation type, and the per-observation action arity. A behavioral profile is a per-player function $\mathsf{InfoState}_i \to \mathrm{PMF}(\mathsf{A}_i)$; a mixed profile is a per-player distribution over the deterministic functions $\mathsf{InfoState}_i \to \mathsf{A}_i$.[2]

Theorem 7.6 (Kuhn behavioral → mixed)**.** *Under the bounded-horizon hypothesis (no infinite traces) and the* no-non-trivial info-state repeat *hypothesis, every behavioral profile β admits an equivalent mixed profile $\mu(\beta)$ in the sense*

$$\mathrm{evalDist}(\beta) \;=\; \mu(\beta) \gg\!= \mathrm{evalPure}.$$

Theorem 7.7 (Kuhn mixed → behavioral)**.** *Under per-step semantic locality assumptions (sufficient for the* ActionPosteriorLocal *property), every mixed profile μ admits an equivalent behavioral profile $\beta(\mu)$ in the sense*

$$\mathrm{evalDist}(\beta(\mu)) \;=\; \mu \gg\!= \mathrm{evalPure}.$$

The two directions are not symmetric. $B \to M$ does not need recall — the mixed lottery can be constructed ex-ante by independent product across information states — but does need horizon-separation (which holds on snapshot-refined $\mathsf{InfoState}$'s). $M \to B$ requires the per-step recall property that, given a player's information state, the conditional distribution of their action under μ does not depend on information *outside* the player's recall. This is strictly weaker than perfect recall in the textbook sense: it does not require remembering one's own past observations literally, only that the action conditional has the right locality.

The snapshot-refined information-state structure is packaged as a derived layer on top of an arbitrary observation model. A second proof of the $M \to B$ direction is given via the mixed-Nash-induced correlated distribution. Language-specific Kuhn theorems for EFG, MAID, MR, and FOSG follow by composing the language's compilation to the observation-model layer with the generic theorem.

Tradeoff. Stating Kuhn on the observation-model layer rather than on a specific syntactic class of EFGs is the single largest piece of abstraction work in the library. The benefit is that one proof serves every language with a generic compilation through that layer; the cost is a stack of technical infrastructure (information-state snapshots, the locality predicates) that is not in the textbook proof. We judge the tradeoff to be sharply positive: language-specific Kuhn theorems become single-line corollaries of a single core result.

Lean correspondence. `Theorems/Kuhn/` contains `ObsModel` (the snapshot-refined wrapper), `ObsModelCore`, `KuhnModel`, the two proof files `BehavioralToMixedCore` and `MixedToBehavioralCore`, and `CorrelatedRealization` (the alternative $M \to B$ route). The hypothesis predicates `NoNontrivialInfoStateRepeat`, `HorizonSeparation`, `ActionPosteriorLocal`, `PerStepPlayerRecall` live on `ObsModelCore`. Supplement §6.5.

[2] Notational reconciliation: the Lean code uses `runDist`/`runDistPure` for the profile-evaluation function on the observation-model carrier; we write evalDist throughout the chapter for uniformity with the EFG-language naming. The two refer to the same operation specialized to the relevant carrier.

7.6 Welfare and welfare theorems

The library packages the basic welfare analysis: social welfare is well-defined under finiteness, every team-game welfare maximizer is Nash, and the price of stability in a team game is 1. We do not formalize the deeper welfare theorems of mechanism design here — they live in §8.

Lean correspondence. `Concepts/WelfareTheorems.lean`.

8 Mechanism design and auctions

The mechanism-design and auction layers are direct consumers of the kernel-game core. They serve a dual role: they formalize classical results that belong in any game-theory library, and they smoke-test the reusability of the core (every definition below is built on the kernel-game/game-form layer or the solution-concept catalogue, with no representation-specific machinery re-introduced).

8.1 Bayesian games

A *Bayesian game* [30] models situations where players have *private information* that affects payoffs — each player knows their own private state but only has a prior belief over others'. Harsanyi's reduction encodes this private information as a *type* $\theta_i \in \Theta_i$ for each player; types are drawn jointly from a common-prior distribution $\mu \in \mathrm{PMF}(\prod_i \Theta_i)$ known to all players, but only θ_i is observed by player i. A pure strategy is then a contingent plan $\Theta_i \to \mathsf{A}_i$ that prescribes an action for every type the player might turn out to be. Concretely, a finite Bayesian game is the tuple $(\Theta, \mathsf{A}, \mu, u)$ with finite Θ_i, A_i and a payoff $u : \prod_i \Theta_i \times \prod_i \mathsf{A}_i \to \mathcal{I} \to \mathbb{R}$ depending on both the realized type vector and the chosen joint action. Behavioral strategies replace the pure version by $\Theta_i \to \mathrm{PMF}(\mathsf{A}_i)$.

The Bayesian game compiles to a kernel game via the `ofEU` adapter (§3.2): the strategy type is $\Theta_i \to \mathsf{A}_i$, and the outcome kernel is the deterministic point mass at the *ex-ante* expected utility

$$\mathrm{exAnteEU}(B, \sigma, i) \;=\; \sum_{\theta} \mu(\theta)\; u(\theta, \sigma_1(\theta_1), \ldots, \sigma_n(\theta_n))(i),$$

folded into a single payoff vector per profile. The type × action sampling is therefore not a separate sampling step in the kernel game's K but is folded into the EU function — a deliberate use of `ofEU` that lets every EU-specific solution concept on $\mathcal{G}$ apply to Bayesian games without further bookkeeping.

Bayesian Nash equilibrium [30] is then the ordinary IsNash predicate of the compiled kernel game, unfolded as

$$\forall i,\, \forall s' : \Theta_i \to \mathsf{A}_i.\quad \mathrm{exAnteEU}(B, \sigma, i) \;\geq\; \mathrm{exAnteEU}(B, \sigma[i \mapsto s'], i).$$

Ex-post and interim variants are exposed as separate derived definitions: ex-post, no profitable deviation conditional on the realized type vector; interim, no profitable deviation conditional on the deviating player's own type.

Tradeoff. Continuous type spaces and continuous action spaces are the natural setting for much of auction theory beyond sealed-bid mechanisms with discrete bid grids. As discussed in §2.1, this falls outside the discrete-PMF discipline and is not in scope. All Bayesian-game results in the library hold for finite type and action spaces.

Lean correspondence. `Mechanism/BayesianGame.lean` (`BayesianGame`, `exAnteEU`, `toKernelGame`, `BayesNash`, `bayesNash_iff_exAnteEU`). Supplement §7.1.

8.2 Mechanism design and the revelation principle

A *general mechanism* [58] is a type-space-and-action-rule $M = (\Theta, \mathsf{A}, g : \prod_i \mathsf{A}_i \to \mathcal{I} \to \mathbb{R})$; a strategy profile is a tuple $\sigma_i : \Theta_i \to \mathsf{A}_i$. A *direct mechanism* is a mechanism with $\mathsf{A}_i = \Theta_i$; the canonical strategy is the identity. The current Lean definition of Bayesian incentive compatibility is an ex-ante finite condition against constant misreports θ_i', not the full interim textbook condition against arbitrary type-dependent reporting rules.

Theorem 8.1 (Revelation principle)**.** *Let M be a general mechanism with finite types and actions, μ a common prior over types, and σ a Bayesian Nash equilibrium of M. Then the direct mechanism M^{direct} obtained by composing each player's report with σ_i — i.e. $M^{\text{direct}}(\theta) = g(\sigma(\theta))$ — is BIC in the library's constant-misreport sense.*

The proof is the textbook argument restricted to the formalized deviation class: a constant report θ_i' in M^{direct} corresponds to the type-independent deviation $\theta_i \mapsto \sigma_i(\theta_i')$ in M, and the Nash hypothesis on σ rules out profitable deviations.

Lean correspondence. `Mechanism/MechanismDesign.lean` and `Mechanism/RevelationPrinciple.lean` (`GeneralMechanism`, `IsBNE`, `toDirect`, `revelation_principle`). Supplement §7.2.

8.3 Social choice

The library formalizes the basic social-choice setup: a finite set of alternatives, a finite set of voters with strict preferences, and a social welfare function or social choice function. It proves the structural fragments needed downstream (preference profiles, the existence of a social-welfare-maximizing alternative under utilitarian aggregation), but does not formalize the impossibility-theorem chain (Arrow [3], Gibbard–Satterthwaite [23, 71]); these are flagged in §11 as the natural next step.

Lean correspondence. `Mechanism/SocialChoice.lean`.

8.4 Auctions

The auction directory currently contains four formats at different levels of completeness:

- *Vickrey (second-price)* [84]. The highest bidder wins and pays the second-highest bid; truthful bidding is a weakly dominant strategy. The proof is by case analysis on whether i's bid exceeds the maximum other bid; in each case, the EU of bidding v_i matches or exceeds the EU of any other bid.
- *First-price sealed bid* [84]. The highest bidder pays their own bid. The file defines the deterministic first-price game and proves that no single bid is a dominant strategy under the stated non-triviality hypotheses.
- *All-pay auction* [45]. Every bidder pays their own bid regardless of winning. The current formalization does not build a kernel game; it records elementary payoff facts such as overbidding being unprofitable, winner profit, rent dissipation, and a symmetric negative-payoff lemma.
- *Vickrey–Clarke–Groves (VCG)* [17, 25, 84]. Generalizes Vickrey to multi-item, combinatorial settings: each agent's payment equals the externality they impose on others. The library proves dominant-strategy incentive compatibility on a finite set of allocations and finite type spaces.

Vickrey and first-price are direct kernel-game constructions via the `ofEU` adapter. VCG is packaged as a finite setup with an efficient allocation rule and a payment function; all-pay is currently a collection of real-valued payoff lemmas. Solution concepts come from §4 wherever a full kernel game has been constructed.

Tradeoff. Auctions are typically formalized in the literature under continuous bid spaces and continuous valuations; this gives the cleanest expressions for symmetric equilibrium bids and expected revenue formulas. Our discrete formalization is faithful to the strategic content it currently formalizes (truthful bidding is dominant in Vickrey/VCG; first-price has no dominant bid; all-pay payoff inequalities are available) but is not the primary place a researcher would go to study revenue ranking under continuous distributions. We treat the auction layer primarily as a smoke test of the core library; downstream, an extension to continuous auctions over mathlib's measure theory would be the natural next step.

Lean correspondence. `Auctions/Vickrey.lean` (`vickrey_truthful_dominant`), `Auctions/FirstPrice.lean`, `Auctions/AllPay.lean`, `Auctions/VCG.lean`. Supplement §7.3.

9 Discussion: design tradeoffs

This section collects, in one place, the design choices that shaped the library, paired with what each gained and what each cost. We have referred to these tradeoffs locally throughout the chapter; the goal here is to make the global picture explicit so that a downstream user can decide whether the library's contract matches their needs.

9.1 Finite, discrete probability

Choice. Every distribution is a `PMF`; every strategy/outcome carrier is a `Fintype` or carries a `Fintype` hypothesis on the relevant theorem. No σ-algebra is ever invoked.

Gained. Every strategic-form definition is a $\sum$-of-products identity that the library's `simp` set can manipulate. No measurability hypotheses propagate through strategy spaces, profile distributions, or outcome distributions. Equational reasoning under the `PMF` monad is mechanically straightforward. Compilation between languages is a calculation on finite sums and Dirac/Bind compositions.

Lost. The cost of mechanizing measure theory and the catalogue of classical results that fall outside the discrete contract are described in detail in §2.1; we do not repeat them here. What §8 adds is the consequence for downstream consumers: a protocol or auction layer that builds on this library inherits the discrete contract and must either rephrase its objects in finite/discrete shape (e.g. a discretized bid grid, a finite type lattice) or build a parallel measure-theoretic infrastructure on top of mathlib and accept the corresponding proof obligations. In practice this is a hard contract: existing downstream users of the library work in the discrete idiom uniformly, and the library's design does not provide an automatic bridge to a continuous version. An honest reading of §2.1 should leave the consumer with a sense of which classical theorems they can call into and which they cannot.

9.2 Protocol/preference split

Choice. `GameForm` is utility-free; `KernelGame` adds utility. Solution concepts exist in two forms: EU-specific predicates on `KernelGame` and preference-parameterized predicates (`*For`) on `GameForm`.

Gained. Theorems about strategic structure (Kuhn, information-set machinery, the deviation hierarchy) state and prove once at the `GameForm` layer, parametric in any preference preorder. EU is one preference preorder; lexicographic, maximin, or non-EU preferences are other preorders that get the same theorems. The library is therefore reusable for, e.g., ordinal-preference solution concepts without rewriting the strategic skeleton.

Lost. One additional indirection for *generic* users. A user who only ever cares about expected utility writes `G.IsNash` σ and is done; a user who wants to state a generic theorem parametric in the preference must write `G.IsNashFor pref` σ and supply the preference explicitly. Two versions of every solution concept exist (`IsNash`, `IsNashFor`, etc.), connected by `_iff_` bridges; the `simp` set has to know about both. In practice the EU-only ergonomics dominates daily use and the generic version is visible only to library authors stating preference-parametric theorems.

Why. The split is motivated by classical mechanism-design practice [59], where the "social-choice rule + payment scheme = mechanism" decomposition is exactly the protocol/preference split made structural. Once we saw that several classical theorems (dominance $\Rightarrow$ Nash; correlated $\Rightarrow$ coarse correlated; the deviation hierarchy) were preference-agnostic, building them at the `GameForm` layer became the natural design.

9.3 The `*For` style versus typeclass overloading

Choice. Preferences are passed as explicit `Prop`-valued arguments to `*For` solution concepts; they are not registered as typeclass instances on `KernelGame`.

Gained. Several library theorems quantify over two preferences at once (monotonicity of IsNashFor in the preference; Pareto dominance, which takes a $(\succeq, \succ)$ pair; strict-implies-weak bridges; the transport theorem of §3.6, which relates a preference to its pullback along an outcome bijection). With explicit-argument preferences each of these is one line; with a typeclass-based design each would force ad-hoc local instances and `let`-binding tricks at the call site.

Lost. Slight verbosity at the call site for the *generic* user: they write `G.IsNashFor G.euPref` σ instead of `G.IsNash` σ whenever they instantiate the generic version at EU. The EU-only user is unaffected — the EU-specific predicates (`IsNash`, `IsBestResponse`, `IsCorrelatedEq`, ...) are exposed directly and the `_iff_` bridges make round-tripping free for the EU case.

Why. The honest version of the argument is not that typeclasses are categorically wrong here — a `[Preference G]` typeclass with a default `euPref` instance would in fact make the EU-only case ergonomically uniform. Rather, the multi-preference theorems listed above fit explicit-argument predicates naturally and fit typeclasses awkwardly, and the EU-only ergonomic is recovered cheaply by the EU-specific layer. The library does use a typeclass where one is the right fit: `PrefPreorder pref` asserts that a preference is reflexive and transitive — a property of the relation, not a choice between relations, and Lean's inference resolves it without diamond/multi-preference issues.

9.4 Solution concepts on `KernelGame` only

Choice. `IsNash`, `IsDominant`, `IsBR`, `IsCorrelatedEq`, etc. are defined on `KernelGame` and on `GameForm`. They are *not* re-defined on each representation (NFG, EFG, MAID, MultiRound, FOSG); concrete representations bridge to the central definitions via toKernelGame.

Gained. The library has one canonical "`IsNash` of an EFG game" — the `IsNash` of its compiled kernel game. Theorems about Nash on EFGs do not have to be re-proved each time a new EFG-like representation is added; they are corollaries of the core theorems via composition with $\rightsquigarrow$. This is the workhorse of the bridge architecture of §5.7.

Lost. Concrete representations occasionally have a representation-specific solution concept that is more natural in their syntax than in the kernel-game image (subgame-perfect equilibrium for EFGs, perfect Bayesian equilibrium for Bayesian games). These exist as separate definitions (e.g. `EFG.IsSubgamePerfectEq`); the library proves their relation to the central `IsNash` as a theorem rather than building it into the definition.

9.5 Noncomputable definitions and classical reasoning

Choice. Many definitions are `noncomputable`, particularly those that invoke real-number operations, the classical disjunction `Function.update` on abstract index types, or fixed-point theorems. We do not attempt to expose computable witnesses for, e.g., `mixed_nash_exists`.

Gained. Proofs are shorter and more uniform. Classical reasoning is locally invoked via `open Classical in` and is flagged at every use. Real-number arithmetic uses mathlib's full real-number API without restriction.

Lost. The library is not a Nash-equilibrium algorithm. A `#eval`-style execution will not produce a witness from `mixed_nash_exists`. Constructive consumers must write their own algorithm and prove it converges to a Nash equilibrium of the same kernel game.

Why. The library targets formalization of theorems, not algorithm certification. Brouwer's fixed-point theorem is inherently non-constructive; replacing it with a constructive substitute (Sperner's lemma) would change the proof shape of Theorem 7.1 without changing the statement, and would not improve the user's experience unless they happened to be running the Lean kernel as an evaluator. We treat constructive existence as a separate, downstream concern.

9.6 Distributional, not deterministic, EFG semantics

Choice. `GameTree.evalDist` returns a `PMF` $\mathcal{O}$ given a *behavioral* profile. No deterministic "play" function exists at the EFG level; pure strategies are lifted to behavioral via point-mass distributions.

Gained. Chance nodes integrate without ad-hoc adapters. Kuhn's theorem becomes a structural identity $\text{evalDist}(\beta) = (\text{realize}(\mu)) \gg\!= \text{evalDist}^{\text{pure}}$ rather than a probabilistic equivalence, and the full game-form toGameForm has the $K : \prod_i \mathsf{S}_i \to \text{PMF}(\mathcal{O})$ shape natively.

Lost. Reasoning about a single play of an EFG (a single deterministic execution path) requires unfolding a `PMF`. This is a wash: traditional textbook reasoning quietly assumes determinism while writing distributional formulas, so making the distributional semantics primary forces explicit bookkeeping in places where the textbook is implicit.

9.7 Fin n player types in bridges

Choice. Bridge theorems often canonicalize the player type to Fin n for the appropriate n.

Gained. A canonical concrete carrier on which to state the bridge. Two bridges into and out of Fin n compose without re-indexing in the middle.

Lost. Source-game player types (e.g. Bool for two-player games) must be transported across an equivalence `Bool` $\simeq$ `Fin 2` to use the bridge.

Why. `KernelGame.reindex` (§3.5) makes the transport a one-line operation. Treating Fin n as the canonical finite-cardinality witness lets every bridge avoid having to pick one for itself.

9.8 What we did not formalize and why

For honesty's sake we list what is missing.

- *Repeated and stochastic games at infinite horizon.* The library's repeated games are bounded. The folk-theorem-style results (Friedman, Fudenberg–Maskin) require infinite or discounted horizons, which interact with measure theory and which we have not formalized.
- *Continuous strategy / type / outcome spaces.* See §9.1.
- *Computational complexity.* Nash existence is formalized; PPAD-completeness [18] is not. Approximate-equilibrium algorithmic results are not formalized.
- *Cooperative game theory beyond Shapley values.* The library defines the Shapley value and proves null-player, additivity, linearity, and symmetry lemmas. The full uniqueness axiomatization, the core, the nucleolus, balanced collections, and the Bondareva–Shapley theorem [14, 78] are not formalized.
- *Impossibility theorems in social choice.* Arrow, Gibbard–Satterthwaite, and Sen's liberal paradox are formalizable in this framework but are not in the current library.
- *Bayesian persuasion and information design.* Kamenica–Gentzkow [38] and successors are out of scope.
- *Mean-field / population games.* These require the continuum and are out of scope.

These are not bugs; they are the boundary of the library's contract.

10 Related work

We organize related work along three axes: the textbook tradition the library mechanizes, prior efforts to mechanize fragments of game theory in interactive theorem provers, and adjacent formalizations in dependent type theory or higher-order logic that could plausibly serve as alternative platforms.

10.1 Textbook game theory

The library's substantive content is drawn from the standard textbook canon. von Neumann and Morgenstern [86] laid the basis for the expected-utility decision theory the library uses without modification; the minimax theorem first appears in von Neumann [85]. Nash [60] and Nash [62] are the sources for the existence theorem in the form proved in Theorem 7.1. Aumann [6, 8] introduce correlated equilibrium and its EU equivalence with incentive-compatibility under a correlation device. Selten [73, 74] introduce subgame perfection and the trembling-hand variants the one-shot deviation principle formalizes. Harsanyi [30] formalizes Bayesian games and type-conditional strategies. Kuhn [46] is the source of the behavioral-vs-mixed equivalence, originally proved under perfect recall; Aumann [5] sharpened the recall hypothesis.

For comprehensive surveys we draw on Osborne and Rubinstein [64], Maschler et al. [51], Shoham and Leyton-Brown [79], and Myerson [59]. The mechanism-design layer follows Myerson's revelation-principle exposition; the auction layer follows Krishna [44]. Algorithmic game theory, including the price of anarchy, is treated in Nisan et al. [63], with Roughgarden's smoothness framework [69, 70] and Koutsoupias–Papadimitriou's original PoA [42] both relevant.

The factored-observation stochastic-game formalism [43] is a recent unification of extensive-form games, partially observable Markov games, and multi-agent reinforcement-learning settings; the library uses it as the most general native presentation, while keeping EFG and NFG as distinct languages because the textbook tradition treats them so. The sequence-form representation of Koller et al. [41] is conspicuously absent from the library; we discuss this in §11.

Multi-agent influence diagrams are due to Koller and Milch [40] and have been further developed in Pfeffer and Gal [66] and Hammond et al. [27]. The strategic-relevance / d-separation analyses of MAIDs are partly formalized in our library (`MAID/FrontierLemmas.lean`, the order-independence content), and partly left as an explicit extension target.

10.2 Prior formalizations of game theory

Coq. Vestergaard [83] gives a Coq-formalized constructive proof of sequential Nash equilibria. Le Roux et al. [47] formalize a Nash-equilibrium existence theorem in both Coq and Isabelle, taking a constructive route distinct from our Brouwer-based proof. Bertot [12] formalized the Stable Marriage / Gale–Shapley algorithm in Coq.

Isabelle/HOL. Lange, Caminati, Kerber, Rowat, and collaborators have produced a substantial body of mechanized auction theory in Isabelle, including VCG, combinatorial auctions, and revenue-equivalence fragments [15, 39]. Their setting is more directly continuous than ours, since Isabelle's analysis library makes real-valued bid spaces ergonomic. Isabelle's CryptHOL [50] formalizes discrete probability and probabilistic programs and has been used for cryptographic-protocol verification rather than game-theoretic equilibria.

Lean. The discrete-probability monad `PMF`, the standard simplex and its convex/topological API, finite-type infrastructure, and the surrounding analysis the library depends on are all in mathlib [52]; the Brouwer fixed-point theorem itself, however, is not in mathlib at the time of writing, and is supplied by the standalone `fixed-point-theorems-lean4` package [28]. Outside the present library, scattered Lean game-theory developments exist (mostly in the form of self-contained proofs of Nash existence for finite games and zero-sum minimax), but to our knowledge none constitute a library on the scale of the present chapter. To our knowledge no other Lean library covers the

protocol/preference split, the language layer, the bridges, the Kuhn theorem at the observation-model level, or the mechanism-design layer at the level we have.

Other systems. ACL2, HOL4, and Mizar each have isolated game-theoretic results we are aware of but no substantial library; we omit them for space.

10.3 Verified probability and probabilistic programming

The library's reliance on `PMF` places it in a tradition of discrete probability formalizations. Hurd et al. [33] and Audebaud and Paulin-Mohring [4] formalized discrete probabilistic programs in Coq with semantic foundations that anticipate the discrete Giry monad. Tassarotti and Harper [82] mechanized the formal-semantics side of higher-order probabilistic programming in Lean and Coq. Lochbihler [50] (CryptHOL) does similar work in Isabelle, with cryptographic applications. Mathlib's `PMF` [52] provides the API on which our library builds.

The measure-theoretic side is more developed in mathlib than the discrete side, with substantial integration across the Lean Borel, Lebesgue, and product-measure infrastructures; an extension of the present library over mathlib's measure theory would draw on this.

10.4 Mechanized programming-language semantics

The library's language layer borrows several patterns from mechanized programming-language semantics: structural operational semantics, denotational compilation into a common target, simulation/refinement [2, 10, 29, 56], and CompCert-style verified-translation pipelines [49]. The "language as syntax + compiler to a common semantics" framing is direct PL practice; what is unusual in our setting is that the common semantics is a strategic object (a kernel game) rather than a behavioral one.

10.5 Categorical game theory

A growing body of work treats games and strategies as objects in a category with a compositional algebra of game forms. Hyland [34], Pavlović [65] are early examples; Ghani et al. [22] introduce *open games* and the bidirectional "selection function" approach; Joyal [35] pioneered the categorical view of two-player games (in a logical setting). Our protocol-map and simulation infrastructure is inspired by this tradition without committing to any of its specific frameworks. An open-games-as-coalgebras approach would be a natural reformulation; we did not take it because the kernel-game record is already small enough that adding a categorical superstructure would, on net, cost ergonomics.

10.6 Mechanized formal verification of equilibria in protocols

A different line of work mechanizes equilibrium reasoning about cryptographic and economic protocols directly. Abraham et al. [1], Halpern and Pass [26] and successors formalize rationality-based protocol analysis and incentive-compatible distributed algorithms. These are downstream consumers of a library like ours; they typically commit to one specific representation of a game (often a payoff matrix or an EFG-like tree) and prove protocol-specific results. Our library is meant to make their job easier by supplying the strategic-form infrastructure they would otherwise build by hand.

11 Future directions

The library is intentionally a foundation: it is meant to be extended. We list the extensions we view as natural, in descending order of how much existing infrastructure they would re-use.

11.1 Algorithmic and constructive Nash

Theorem 7.1 is non-constructive (Brouwer); a natural extension is a constructive proof for two-player games via the *Lemke–Howson* pivoting algorithm [48] or via *Sperner's lemma* [81] (a combinatorial fixed-point statement on triangulated simplices that yields Brouwer constructively for low dimensions), packaged as an algorithmic witness function on top of the existing strategic-form definitions. The *PPAD*-hardness story [18] — where PPAD is the complexity class of problems whose existence is guaranteed by a parity argument on a directed graph, the canonical example being end-of-line — is at a different level (it concerns problem reductions, not proof-of-existence) and is largely orthogonal: a complexity formalization would need an independent computation model.

11.2 Sequence form and efficient EFG solving

A *sequence* in an extensive-form game with perfect recall is the list of own-actions a player has taken along a history. The *sequence-form representation* of Koller et al. [41] (see also the survey of von Stengel [87]) replaces the exponential-sized pure strategy space by realization probabilities indexed by sequences, turning equilibrium computation into a linear or quadratic program of size polynomial in the tree. Adding the sequence-form bridge as a fourth EFG strategy carrier (alongside pure, behavioral, mixed) would let the library state and prove correctness of the standard solving algorithms (LP duality⇒two-player zero-sum value; sequence-form quadratic program for general-sum games). The Kuhn-theorem infrastructure of §7.5 already supplies the recall layer.

11.3 Counterfactual regret minimization

Counterfactual regret minimization (CFR, 88) is the dominant family of algorithms for finding approximate equilibria in large imperfect-information extensive-form games; it works by tracking, at each information set, the cumulative regret for not having played each action, counterfactually weighted by the probability of reaching that information set under the opponents' play. The regret-equilibrium duality is already formalized in the library (§4.7), making the CFR convergence proof a natural consumer. This would also produce a constructive CCE existence theorem (§7.2) without going through Brouwer.

11.4 Imperfect-recall extensions

The Kuhn theorem in the library handles a recall hypothesis weaker than perfect recall (per-step ActionPosteriorLocal). A natural extension is to formalize the *absent-minded-driver* phenomenon [67] — a single-player imperfect-recall puzzle where a player who forgets whether they have already made one decision faces a time-consistency problem (the ex-ante optimal mixed strategy is not the same as the strategy the player would choose at the decision point if they could) — and the broader family of imperfect-recall pathologies it generates. These require exactly the representation-neutral observation model the library already exposes.

11.5 Infinite and continuous games

The largest single extension would be to relax the discrete-PMF discipline and use mathlib's measure theory. This would let the library state and prove: Glicksberg's theorem on continuous payoff functions over compact strategy spaces [24], the Milgrom–Weber existence theorem for Bayesian games with uncountable type spaces [55], infinite-horizon discounted stochastic games [54, 77], and the folk theorems for repeated games. Several theorems of the library would then have continuous companions.

This is a substantial extension and would change the library's character. We see it less as a single project and more as a parallel library with shared infrastructure (the `KernelGame`/`GameForm` skeleton generalizes to a kernel $K : \prod_i \mathsf{S}_i \to \mathcal{P}(\mathcal{O})$ for a measure type $\mathcal{P}$, with the discrete `PMF` as one instantiation).

11.6 Cooperative game theory

The Shapley value definition and its basic null-player, additivity, linearity, and symmetry lemmas (`NFG/CoalitionalGame.lean`) are in the library. Natural extensions: the full uniqueness axiomatization, the core, balanced collections [14, 78], the nucleolus [72], the Aumann–Shapley value for non-atomic games [9], and bargaining solutions beyond Nash's [36, 37]. The TU coalitional structure is already there; only the solution-concept layer needs to be filled in.

11.7 Information design and signaling

In *Bayesian persuasion* [38] a sender designs an information structure (a signal-generation mechanism that maps the state of the world to a public message) to influence a receiver's posterior beliefs and hence their action; the design problem is to choose the most persuasive signal mechanism subject to the receiver's Bayesian rationality. *Signaling games* [80] are the strategic analogue with an informed sender choosing a costly signal whose cost may itself depend on type. *Information design* [11] generalizes the persuasion setup to multiple receivers and richer outcome rules. All three sit on top of the Bayesian-game infrastructure of §8.1 and would benefit from a continuous-type extension, but admit interesting finite-grid formalizations directly.

11.8 Social choice impossibility theorems

Arrow's impossibility theorem [3] says no social-welfare function aggregating individual ordinal preferences over ≥ 3 alternatives can simultaneously satisfy unanimity, non-dictatorship, and independence of irrelevant alternatives. *Gibbard–Satterthwaite* [23, 71] is the strategic analogue: any non-dictatorial social choice function with full range over ≥ 3 alternatives is manipulable. *Sen's liberal paradox* [75] shows that even a minimal individual-rights condition is inconsistent with Pareto efficiency plus universal domain. All three are statable on the existing social-choice setup, the proofs are combinatorial and do not require probability, and they would fit the library cleanly.

11.9 Multi-agent learning

The FOSG and observation-model layers are built precisely so that multi-agent reinforcement-learning algorithms — *fictitious play* (each player best-responds to the empirical distribution of opponents' past actions), self-play, MCTS, regret matching — can be formalized as strategy-update rules on top of an existing game. *No-regret learning* is the central convergence notion: an algorithm has *vanishing average regret* when its average per-round payoff approaches that of the best fixed

action in hindsight; the classical theorems [13, 16] link no-regret play of a repeated game to convergence to CCE (external regret) or CE (internal regret). These are the natural formal targets; the strategic-form definitions they need are already in place.

11.10 Stable matching and Gale–Shapley

The library's NFG layer formalizes the matching predicates (`IsMatching`, `IsBlockingPair`, `IsStable`) and a few structural lemmas, but not the Gale–Shapley deferred-acceptance algorithm itself nor the existence of a stable matching from an arbitrary preference profile [21]. The algorithm is constructive, terminates in $O(n^2)$ steps, and is a natural target for a Lean implementation paired with a correctness proof. Bertot's Coq formalization [12] is a useful reference point.

11.11 Symmetric Nash existence

Existence of a *symmetric* mixed Nash equilibrium in a finite symmetric game [62] is a classical corollary of mixed Nash existence (Theorem 7.1) by restricting the gain map to the symmetric subspace of the product simplex. The restriction step is straightforward but is not packaged on top of `SymmetricGame.lean`.

11.12 Type-theoretic refactoring opportunities

We close with two design directions internal to the library itself. First, an open-games-style reformulation [22] would expose the `KernelGame` category in a shape suitable for compositional reasoning at a higher abstraction level; this would not change the theorems but might simplify their proofs. Second, the library's `noncomputable` discipline could be tightened by isolating the constructive fragment (definitions and theorems that do not invoke classical reasoning) as a separately-named namespace; this would make the boundary between the constructive finite-game fragment and the non-constructive existence-of-Nash fragment visible in user code.

References

[1] Ittai Abraham, Danny Dolev, Rica Gonen, and Joseph Y. Halpern. Distributed computing meets game theory: robust mechanisms for rational secret sharing and multiparty computation. In *PODC*, 2006.

[2] Samson Abramsky and Achim Jung. Domain theory. In *Handbook of Logic in Computer Science, Vol. 3*. Oxford University Press, 1994.

[3] Kenneth J. Arrow. *Social Choice and Individual Values*. Wiley, 1951.

[4] Philippe Audebaud and Christine Paulin-Mohring. Proofs of randomized algorithms in Coq. In *Science of Computer Programming*, 2009.

[5] Robert J. Aumann. Mixed and behavior strategies in infinite extensive games. *Annals of Mathematics Studies*, 52:627–650, 1964.

[6] Robert J. Aumann. Subjectivity and correlation in randomized strategies. *Journal of Mathematical Economics*, 1(1):67–96, 1974.

[7] Robert J. Aumann. Agreeing to disagree. *Annals of Statistics*, 4(6):1236–1239, 1976.

[8] Robert J. Aumann. Correlated equilibrium as an expression of Bayesian rationality. *Econometrica*, 55(1):1–18, 1987.

[9] Robert J. Aumann and Lloyd S. Shapley. *Values of Non-Atomic Games*. Princeton University Press, 1974.

[10] Nick Benton, Chung-Kil Hur, Andrew Kennedy, and Conor McBride. Strongly typed term representations in Coq. In *Journal of Automated Reasoning*, 2014.

[11] Dirk Bergemann and Stephen Morris. Information design: a unified perspective. *Journal of Economic Literature*, 57(1):44–95, 2019.

[12] Yves Bertot. Formalising the Gale–Shapley algorithm in Coq. In *ITP*, 2016.

[13] Avrim Blum and Yishay Mansour. From external to internal regret. *Journal of Machine Learning Research*, 8:1307–1324, 2007.

[14] Olga N. Bondareva. Some applications of linear programming methods to the theory of cooperative games. *Problemy Kibernetiki*, 10:119–139, 1963.

[15] Marco Caminati, Manfred Kerber, Christoph Lange, and Colin Rowat. VCG – a verified combinatorial auction in Isabelle/HOL. In *Workshop on Mechanism Design & Auctions*, 2014.

[16] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.

[17] Edward H. Clarke. Multipart pricing of public goods. *Public Choice*, 11:17–33, 1971.

[18] Constantinos Daskalakis, Paul W. Goldberg, and Christos H. Papadimitriou. The complexity of computing a Nash equilibrium. *SIAM Journal on Computing*, 39(1):195–259, 2009.

[19] James W. Friedman. A non-cooperative equilibrium for supergames. *Review of Economic Studies*, 38:1–12, 1971.

[20] Drew Fudenberg and Eric Maskin. The folk theorem in repeated games with discounting or with incomplete information. *Econometrica*, 54:533–554, 1986.

[21] David Gale and Lloyd S. Shapley. College admissions and the stability of marriage. *American Mathematical Monthly*, 69:9–15, 1962.

[22] Neil Ghani, Jules Hedges, Viktor Winschel, and Philipp Zahn. Compositional game theory. *LICS*, 2018.

[23] Allan Gibbard. Manipulation of voting schemes: a general result. *Econometrica*, 41:587–601, 1973.

[24] Irving L. Glicksberg. A further generalization of the Kakutani fixed point theorem with application to Nash equilibrium points. *Proceedings of the American Mathematical Society*, 3(1): 170–174, 1952.

[25] Theodore Groves. Incentives in teams. *Econometrica*, 41:617–631, 1973.

[26] Joseph Y. Halpern and Rafael Pass. Game theory with costly computation. *Innovations in Theoretical Computer Science*, 2008.

[27] Lewis Hammond, James Fox, Tom Everitt, Ryan Carey, Alessandro Abate, and Michael Wooldridge. Reasoning about causality in games. *Artificial Intelligence*, 320:103919, 2023. doi: 10.1016/j.artint.2023.103919.

[28] harfe. Fixed-point theorems in Lean 4: Brouwer and Kakutani. Lean 4 package, `https://github.com/harfe/fixed-point-theorems-lean4`, 2024–. Cubical Sperner's-lemma route after Kuhn (1960).

[29] Robert Harper. *Practical Foundations for Programming Languages*. Cambridge University Press, 2 edition, 2016.

[30] John C. Harsanyi. Games with incomplete information played by "Bayesian" players, parts I–III. *Management Science*, 14:159–182, 320–334, 486–502, 1967.

[31] Sergiu Hart and Andreu Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica*, 68(5):1127–1150, 2000.

[32] Benjamin Heymann, Michel De Lara, and Jean-Philippe Chancelier. Kuhn's equivalence theorem for games in intrinsic form, 2020.

[33] Joe Hurd, Annabelle McIver, and Carroll Morgan. Probabilistic guarded commands mechanized in HOL. *Theoretical Computer Science*, 346(1):96–112, 2005.

[34] J. M. E. Hyland. Some reasons for generalising domain theory. *Mathematical Structures in Computer Science*, 2010.

[35] André Joyal. Remarques sur la théorie des jeux à deux personnes. *Gazette des Sciences Mathématiques du Québec*, 1(4):46–52, 1977.

[36] Ehud Kalai. Proportional solutions to bargaining situations. *Econometrica*, 45:1623–1630, 1977.

[37] Ehud Kalai and Meir Smorodinsky. Other solutions to Nash's bargaining problem. *Econometrica*, 43(3):513–518, 1975.

[38] Emir Kamenica and Matthew Gentzkow. Bayesian persuasion. *American Economic Review*, 101(6):2590–2615, 2011.

[39] Manfred Kerber, Christoph Lange, and Colin Rowat. An introduction to mechanized auction theory. In *Proceedings of the ITP workshop on auctions*, 2016.

[40] Daphne Koller and Brian Milch. Multi-agent influence diagrams for representing and solving games. *Games and Economic Behavior*, 45:181–221, 2003.

[41] Daphne Koller, Nimrod Megiddo, and Bernhard von Stengel. Fast algorithms for finding randomized strategies in game trees. *STOC*, pages 750–759, 1994.

[42] Elias Koutsoupias and Christos Papadimitriou. Worst-case equilibria. In *STACS*, 1999.

[43] Vojtěch Kovařík, Martin Schmid, Neil Burch, Michael Bowling, and Viliam Lisý. Rethinking formal models of partially observable multi-agent decision making. *Artificial Intelligence*, 303, 2022.

[44] Vijay Krishna. *Auction Theory*. Academic Press, 2 edition, 2010.

[45] Vijay Krishna and John Morgan. An analysis of the war of attrition and the all-pay auction. *Journal of Economic Theory*, 72:343–362, 1997.

[46] Harold W. Kuhn. Extensive games and the problem of information. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games, Vol. II*, pages 193–216. Princeton University Press, 1953.

[47] Stéphane Le Roux, Érik Martin-Dorel, and Jan-Georg Smaus. An existence theorem of Nash equilibrium in Coq and Isabelle. In *Proceedings of the Eighth International Symposium on Games, Automata, Logics and Formal Verification*, volume 256 of *Electronic Proceedings in Theoretical Computer Science*, pages 46–60, 2017. doi: 10.4204/EPTCS.256.4.

[48] Carlton E. Lemke and Joseph T. Howson. Equilibrium points of bimatrix games. *Journal of the Society for Industrial and Applied Mathematics*, 12:413–423, 1964.

[49] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7): 107–115, 2009.

[50] Andreas Lochbihler. Probabilistic functions and cryptographic oracles in higher-order logic. In *ESOP*, 2016.

[51] Michael Maschler, Eilon Solan, and Shmuel Zamir. *Game Theory*. Cambridge University Press, 2013.

[52] The mathlib Community. The Lean mathematical library, 2020–. `https://leanprover-community.github.io/`.

[53] John Maynard Smith and George R. Price. The logic of animal conflict. *Nature*, 246:15–18, 1973.

[54] Jean-François Mertens and Abraham Neyman. Stochastic games. *International Journal of Game Theory*, 10:53–66, 1981.

[55] Paul R. Milgrom and Robert J. Weber. Distributional strategies for games with incomplete information. *Mathematics of Operations Research*, 10:619–632, 1985.

[56] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.

[57] Dov Monderer and Lloyd S. Shapley. Potential games. *Games and Economic Behavior*, 14: 124–143, 1996.

[58] Roger B. Myerson. Optimal auction design. *Mathematics of Operations Research*, 6:58–73, 1981.

[59] Roger B. Myerson. *Game Theory: Analysis of Conflict*. Harvard University Press, 1991.

[60] John F. Nash. Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1):48–49, 1950.

[61] John F. Nash. The bargaining problem. *Econometrica*, 18(2):155–162, 1950.

[62] John F. Nash. Non-cooperative games. *Annals of Mathematics*, 54(2):286–295, 1951.

[63] Noam Nisan, Tim Roughgarden, Éva Tardos, and Vijay V. Vazirani, editors. *Algorithmic Game Theory*. Cambridge University Press, 2007.

[64] Martin J. Osborne and Ariel Rubinstein. *A Course in Game Theory*. MIT Press, 1994.

[65] Duško Pavlović. A semantical approach to equilibria and rationality. *CALCO*, 2009.

[66] Avi Pfeffer and Ya'akov Gal. On the reasoning patterns of agents in games. In *AAAI*, 2007.

[67] Michele Piccione and Ariel Rubinstein. On the interpretation of decision problems with imperfect recall. *Games and Economic Behavior*, 20:3–24, 1997.

[68] Robert W. Rosenthal. A class of games possessing pure-strategy Nash equilibria. *International Journal of Game Theory*, 2:65–67, 1973.

[69] Tim Roughgarden. *Selfish Routing and the Price of Anarchy*. MIT Press, 2005.

[70] Tim Roughgarden. Intrinsic robustness of the price of anarchy. *Journal of the ACM*, 62(5): 32:1–32:42, 2015.

[71] Mark A. Satterthwaite. Strategy-proofness and Arrow's conditions: existence and correspondence theorems for voting procedures and social welfare functions. *Journal of Economic Theory*, 10:187–217, 1975.

[72] David Schmeidler. The nucleolus of a characteristic function game. *SIAM Journal on Applied Mathematics*, 17(6):1163–1170, 1969.

[73] Reinhard Selten. Spieltheoretische Behandlung eines Oligopolmodells mit Nachfrageträgheit. *Zeitschrift für die gesamte Staatswissenschaft*, 121:301–324, 667–689, 1965.

[74] Reinhard Selten. Reexamination of the perfectness concept for equilibrium points in extensive games. *International Journal of Game Theory*, 4:25–55, 1975.

[75] Amartya K. Sen. The impossibility of a Paretian liberal. *Journal of Political Economy*, 78: 152–157, 1970.

[76] Lloyd S. Shapley. A value for n-person games. In H. W. Kuhn and A. W. Tucker, editors, *Contributions to the Theory of Games, Vol. II*, pages 307–317. Princeton University Press, 1953.

[77] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39: 1095–1100, 1953.

[78] Lloyd S. Shapley. On balanced sets and cores. *Naval Research Logistics Quarterly*, 14:453–460, 1967.

[79] Yoav Shoham and Kevin Leyton-Brown. *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Cambridge University Press, 2008.

[80] Michael Spence. Job market signaling. *Quarterly Journal of Economics*, 87:355–374, 1973.

[81] Emanuel Sperner. Neuer Beweis für die Invarianz der Dimensionszahl und des Gebietes. *Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg*, 6:265–272, 1928.

[82] Joseph Tassarotti and Robert Harper. A separation logic for concurrent randomized programs. In *POPL*, 2019.

[83] René Vestergaard. A constructive approach to sequential Nash equilibria. *Information Processing Letters*, 97(2):46–51, 2006. doi: 10.1016/j.ipl.2005.10.005.

[84] William Vickrey. Counterspeculation, auctions, and competitive sealed tenders. *Journal of Finance*, 16:8–37, 1961.

[85] John von Neumann. Zur Theorie der Gesellschaftsspiele. *Mathematische Annalen*, 100:295–320, 1928.

[86] John von Neumann and Oskar Morgenstern. *Theory of Games and Economic Behavior*. Princeton University Press, 1944.

[87] Bernhard von Stengel. Computing equilibria for two-person games. In Robert J. Aumann and Sergiu Hart, editors, *Handbook of Game Theory with Economic Applications, Vol. 3*, pages 1723–1759. Elsevier, 2002.

[88] Martin Zinkevich, Michael Johanson, Michael Bowling, and Carmelo Piccione. Regret minimization in games with incomplete information. In *NeurIPS*, 2007.